SOFTWARE REQUIREMENTS SPECIFICATION

for

AutoTal Software

Version 1.4.7 approved

Prepared for

Tanmay Bharadwaj
Bharadwaj Communication

Prepared by

Team 1 Softee

September 14, 2026

Contents

1 Introduction **5**
- 1.1 Purpose . . . 5
- 1.2 Document Conventions . . . 5
- 1.3 Intended Audience . . . 5
- 1.4 Product Scope . . . 6
- 1.5 References . . . 6

2 Overall Description **6**
- 2.0.1 Domain Layer . . . 7
- 2.0.2 Service Layer . . . 8
- 2.0.3 Data Access Layer . . . 9
- 2.0.4 Import Module . . . 10
- 2.1 Product Perspective . . . 10
- 2.2 Product Functions . . . 12
- 2.3 User Classes . . . 12
- 2.4 Operating Environment . . . 12
- 2.5 Constraints . . . 13
- 2.6 Assumptions . . . 13

3 External Interface Requirements **14**
- 3.1 Use Case Model . . . 14
- 3.2 User Interfaces . . . 15
- 3.3 Software Interfaces . . . 15

4 System Features **15**
- 4.1 Data Import Engine . . . 15
  - 4.1.1 Description . . . 15
  - 4.1.2 Stimulus/Response . . . 16
  - 4.1.3 Functional Requirements . . . 16
- 4.2 Tally Synchronization . . . 18
  - 4.2.1 Description . . . 18
  - 4.2.2 Stimulus/Response . . . 18
  - 4.2.3 Functional Requirements . . . 19
- 4.3 Inventory Management . . . 21
  - 4.3.1 Description . . . 21
  - 4.3.2 Stimulus/Response . . . 21
  - 4.3.3 Functional Requirements . . . 21
- 4.4 Retailer Management . . . 24
  - 4.4.1 Functional Requirements . . . 24
- 4.5 Bank Reconciliation . . . 25
  - 4.5.1 Functional Requirements . . . 25
- 4.6 Stock Statement Generator . . . 26
  - 4.6.1 Functional Requirements . . . 26
- 4.7 Pricing Module . . . 28
- 4.8 Alert System . . . 29
- 4.9 Backup System . . . 30
- 4.10 Access System . . . 30

5 Other Nonfunctional Requirements **30**
5.1 Performance 30
5.2 Reliability 31
5.3 Security 31
5.4 Usability 32
5.5 Maintainability 32
5.6 Scalability 33

6 Business Rules **33**

7 Error Handling Requirements **34**

8 Data Integrity Requirements **34**

9 Appendix A: Glossary **35**

10 Appendix B: Analysis Models **44**
10.1 Data Flow Diagrams 44
10.2 Database Design 45
10.2.1 Entity Relationship Diagram 45
10.2.2 Schema Tables 46
10.3 Data Validation Specification 47
10.3.1 Lava Sales Import Schema 47
10.3.2 Bank Statement Import Schema 47
10.4 Error Code Registry 47
10.5 Requirement Traceability Matrix (Sample) 48

11 Appendix C: Future Enhancements **48**

Revision History

Name	Date	Reason for Changes	Version
Sanchit	Feb 16, 2026	Initial draft with basic structure and preliminary requirements	1.0
Sanidhya	Feb 20, 2026	Added core functional requirements and system feature breakdown	1.1
Sanidhya	Feb 21, 2026	Refined requirements, added use-case alignment and improved clarity	1.2
Lakshay	Mar 11, 2026	Incorporated non-functional requirements and interface specifications	1.3
Sanchit	Mar 17, 2026	Full IEEE expansion: atomic requirements, validation rules, error handling, edge cases, detailed definitions, and final review	1.4

1 Introduction

1.1 Purpose

This document defines the software requirements for AutoTal, a system designed to automate accounting and inventory synchronization between Lava distributor operations and Tally ERP.

Additional context: AutoTal replaces the current manual workflow where business operations are conducted on the Lava distributor portal (mylava / IndLava) and accounting entries are re-entered manually into Tally ERP. The system imports structured CSV/XLS exports, validates them, creates double-entry accounting vouchers through a Draft → Review → Finalise lifecycle, and maintains IMEI-level inventory using the Specific Identification valuation method.

1.2 Document Conventions

- Requirements are uniquely identified (e.g., REQ-INV-1).
- All requirements are atomic, testable, and verifiable.
- Priority levels: High, Medium, Low.
- The word *shall* denotes a mandatory requirement; *should* denotes a recommendation.
- Prefixes: IMP (Import), SYNC (Voucher/Sync), INV (Inventory), RET (Retailer), BANK (Bank), REP (Reports), PRICE (Pricing), ALERT (Alert), BACKUP (Backup), ACC (Access), ERR (Error Handling), INT (Data Integrity).
- All monetary amounts are in INR (Indian Rupee).
- All timestamps follow ISO 8601 (YYYY-MM-DD HH:MM:SS) in IST (UTC+5:30).
- Requirements marked **[CONSTRAINT]** represent hard system boundaries that cannot be relaxed.

1.3 Intended Audience

Developers, testers, project managers, and stakeholders.

Audience	**Relevant Sections**
Software Developers	Sections 3, 4, 5, 6 – functional requirements and system interfaces
QA / Test Engineers	Sections 4, 5 – testable stimulus/response sequences and validation rules
Project Managers	Sections 1, 2, 7 – scope, constraints, and non-functional requirements
Business Stakeholders	Sections 1, 2, 8 – product scope and business rules
System Architects	Sections 2, 3, 4, 5 – architecture constraints and interfaces

1.4 Product Scope

AutoTal eliminates manual data entry, improves accuracy, and enables real-time validation of financial and inventory data.

Additional scope detail: The system operates as a desktop-first, offline-capable modular monolith. It imports Lava portal CSV/XLS exports, maps them to an internal double-entry accounting schema, maintains per-IMEI inventory (Specific Identification valuation), generates bank reconciliation reports and stock statements, and exposes a read-only REST API for mobile/web dashboard access.

Out of Scope (Version 2.0):

- Real-time API integration with Lava portal (no official API exists).
- Direct Tally ERP write-back (planned for Version 3.0).
- Multi-company or multi-distributor support.
- E-commerce or B2C retail operations.

1.5 References

- IEEE SRS Standard (IEEE Std 830-1998)
- Tally ERP Documentation
- ISO 8601:2004 – Data elements and interchange formats (Date and Time)
- GSMA IMEI Standard – 15-digit decimal identifier (TAC + SNR + CD)
- Indian Accounting Standards (Ind AS) applicable to distributor accounting
- Lava Portal CSV Export Format Documentation (Internal – v1.3)
- PostgreSQL 15 Documentation: `https://www.postgresql.org/docs/15/`
- ISO/IEC 7812 – Luhn algorithm for IMEI check digit validation

2 Overall Description

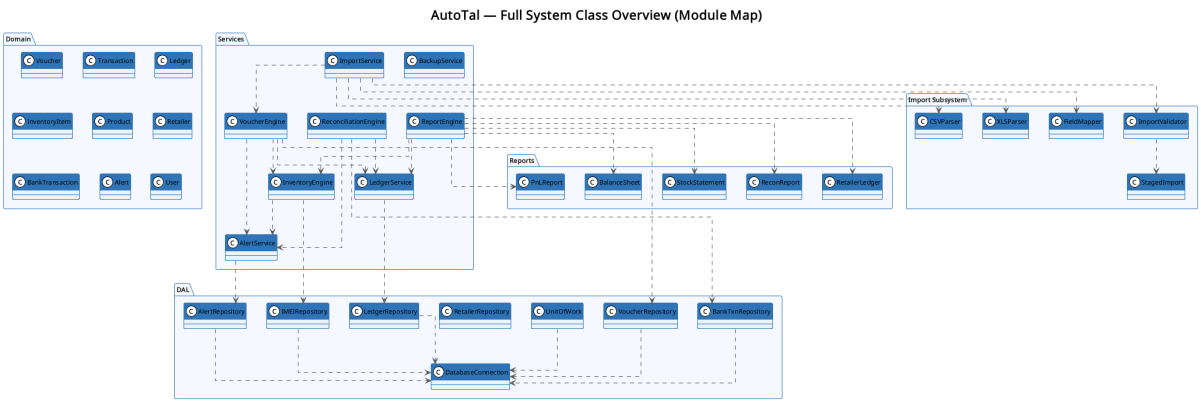


Figure 1: Overall Class Structure

2.0.1 Domain Layer

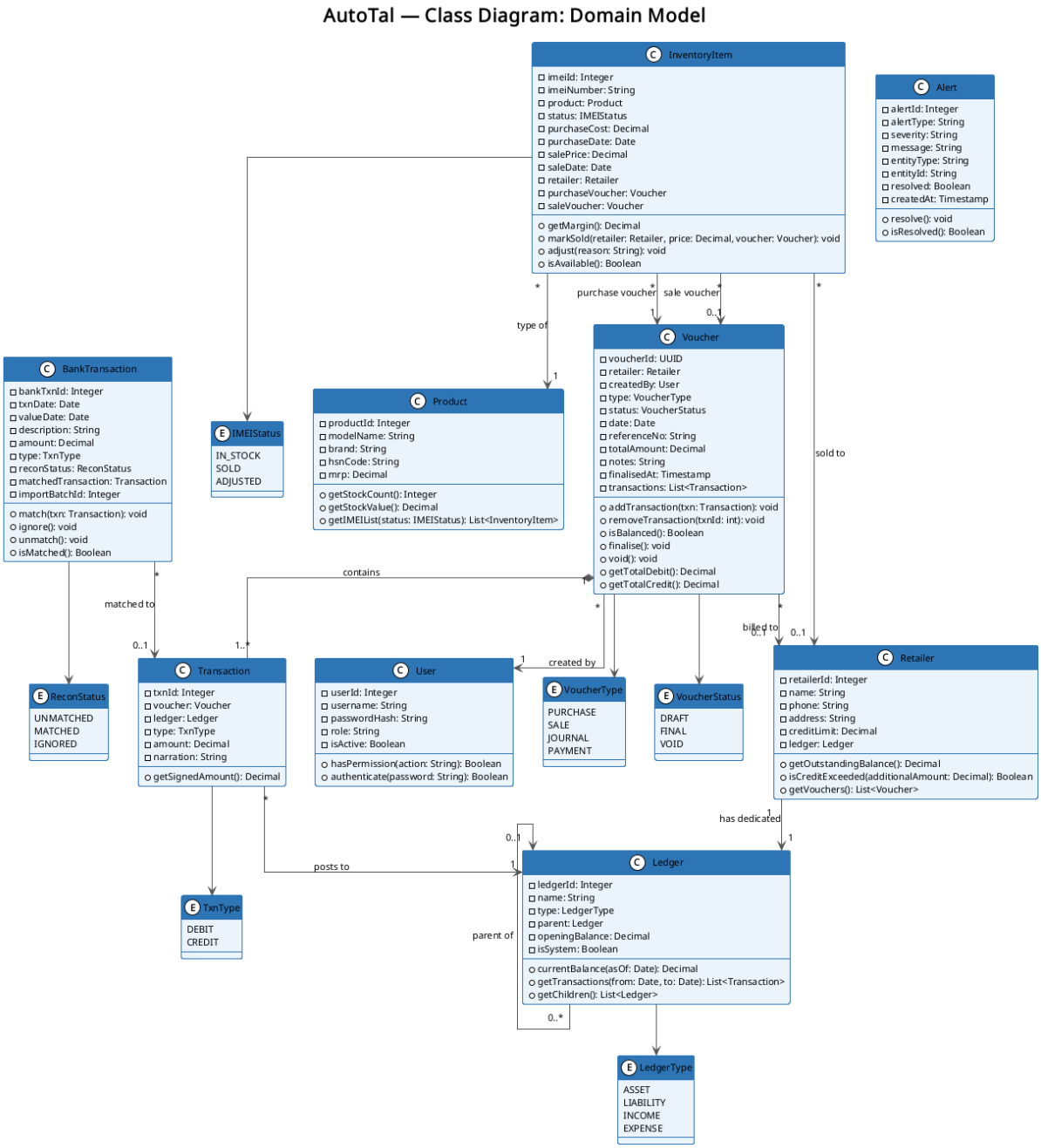


Figure 2: Domain Layer Classes

2.0.2 Service Layer

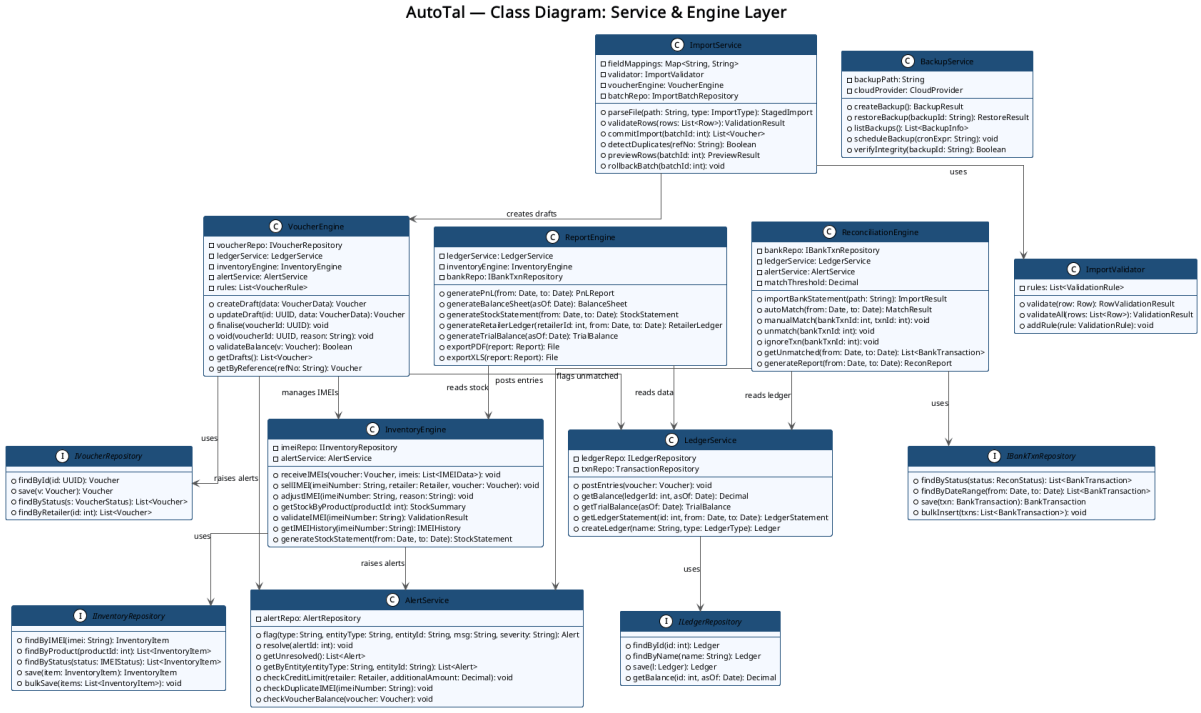


Figure 3: Service Layer Classes

2.0.3 Data Access Layer

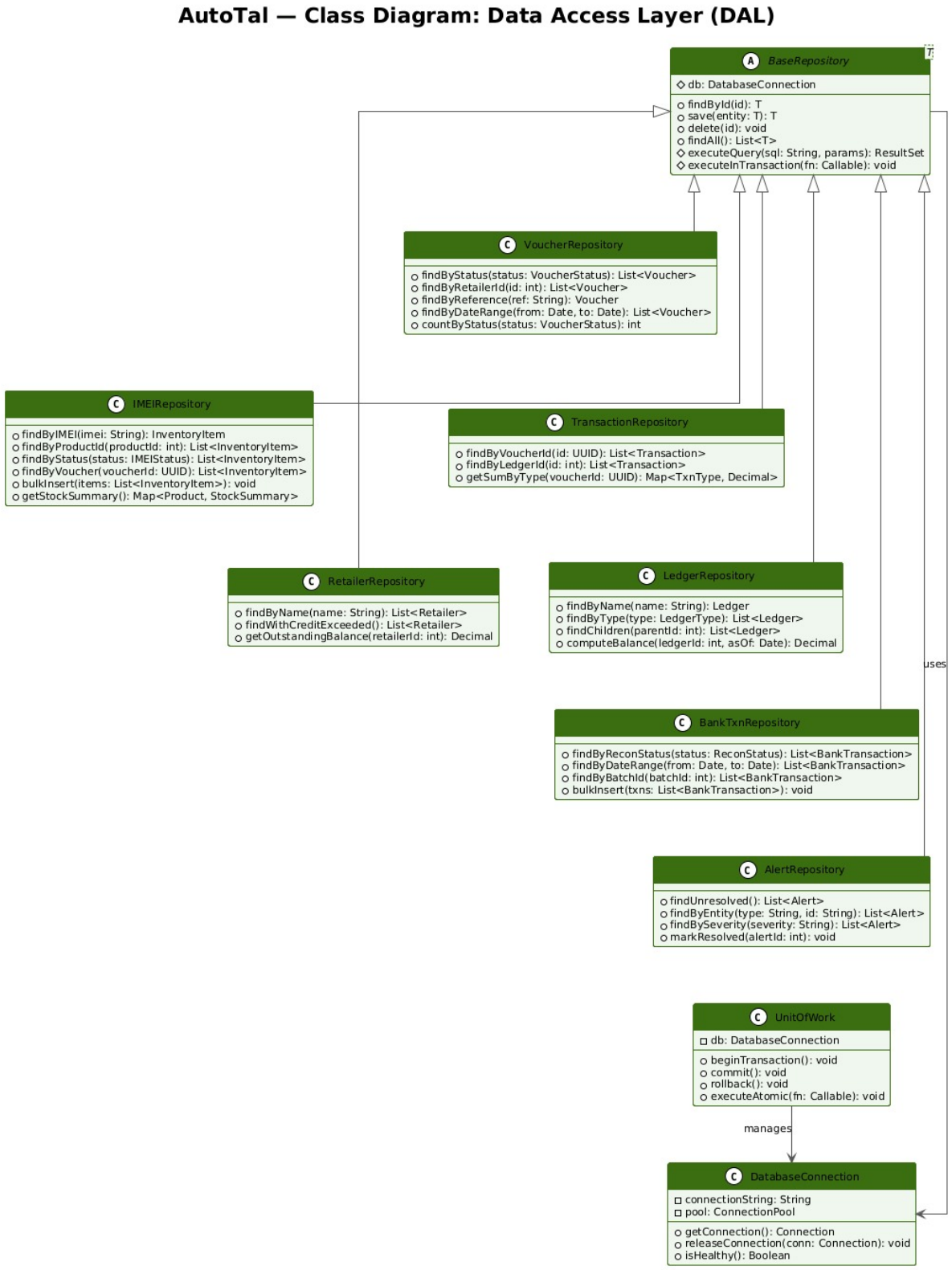


Figure 4: Data Access Layer (Repository Pattern)

2.0.4 Import Module

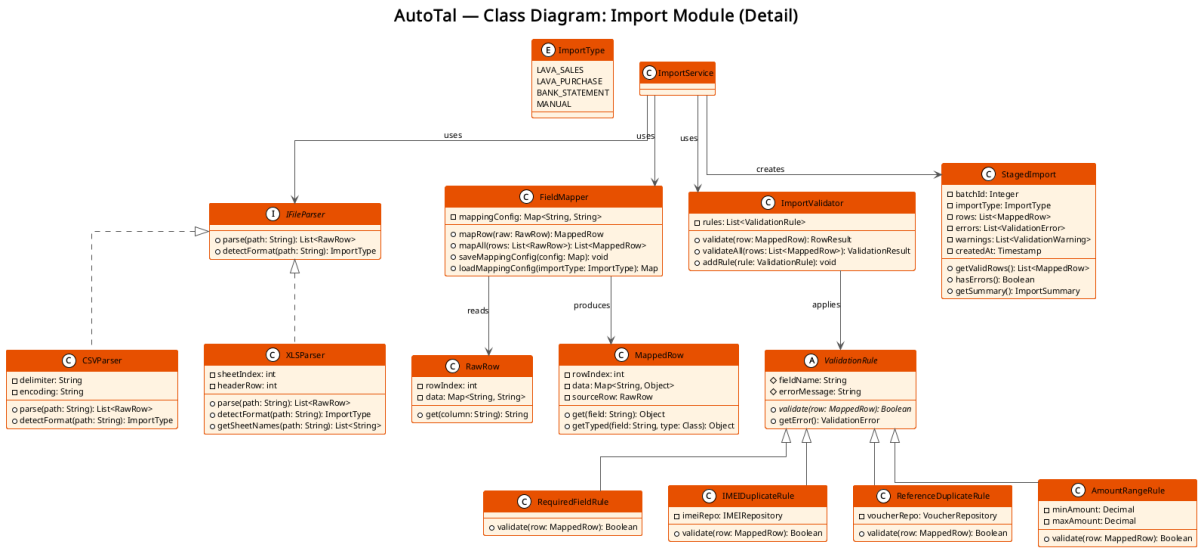


Figure 5: Import Module Classes

2.1 Product Perspective

Standalone desktop system integrating with Tally and processing Lava portal exports.

Additional perspective: AutoTal is structured as a modular monolith – a single deployable unit with clearly bounded internal modules (Import Layer, Voucher Engine, Inventory Engine, Ledger Service, Report Engine, Alert Service, Data Access Layer). A microservices approach is explicitly rejected: the system requires ACID transactions across modules, operates offline, and is maintained by a small team. The Data Access Layer abstracts all database operations through a Repository pattern, supporting both PostgreSQL 15 (production) and SQLite 3.40+ (single-user lite mode) without changing business logic.

#	Module	Primary Function
1	Data Import Engine	Parse, validate, and stage CSV/XLS files from Lava portal and bank
2	Voucher Engine	Create Draft vouchers; enforce DR/CR balance; Finalise atomically
3	Inventory Management	Track every IMEI from purchase to sale; Specific Identification valuation
4	Ledger System	Maintain double-entry ledger; compute balances; generate Trial Balance
5	Retailer Management	Per-retailer ledger, credit limit tracking, outstanding balance
6	Bank Reconciliation	Import bank statements; auto-match by date+amount; flag unmatched
7	Report Engine	Stock Statement, P&L, Balance Sheet, Retailer Ledger, Reconciliation Report

8	Alert System	Non-blocking anomaly detection; severity classification; resolution tracking
9	Backup System	Scheduled and manual backup; AES-256 encrypted storage; one-click restore
10	Access Control	Role-based access (Admin, Accountant); read-only mobile REST API

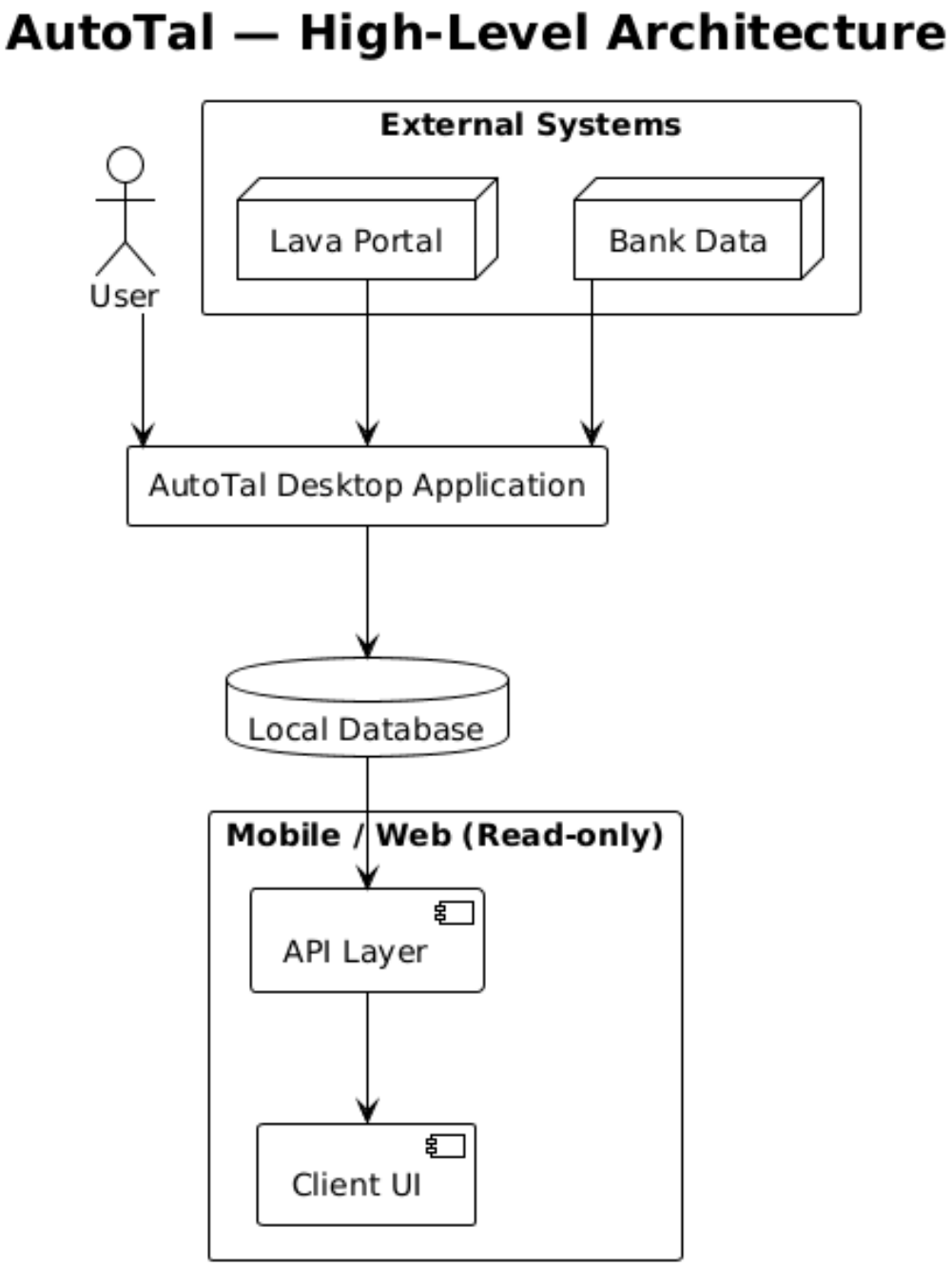


Figure 6: System Architecture of AutoTal

2.2 Product Functions

- Data import and validation
- Tally synchronization
- IMEI-based inventory tracking
- Retailer ledger management
- Bank reconciliation
- Stock reporting

2.3 User Classes

- Distributor (Primary)
- Accountant (Secondary)

User Class	**Description**	**Technical Level**	**Access Scope**
Admin (Owner)	Full system access including configuration, void vouchers, back-up/restore	Low-to-medium	Full
Accountant	Day-to-day operations: import, review, finalise, reports	Medium	Operational (no void, no admin config)
Mobile Viewer	Read-only dashboard and reports via browser/app	Low	Read-only REST API

2.4 Operating Environment

Windows/Linux, Office Laptop, Tally ERP.

- **OS:** Windows 10/11 (primary), Ubuntu 22.04 LTS (secondary)
- **RAM:** Minimum 4 GB; Recommended 8 GB
- **Storage:** Minimum 10 GB free disk space for application and database
- **Database:** PostgreSQL 15 (production) / SQLite 3.40+ (lite mode)
- **Runtime:** Python 3.11+ with PyQt6 or Electron 28+ with Node.js 20 LTS
- **Network:** No continuous internet required for core operations
- **Display:** Minimum 1280×720 resolution

2.5 Constraints

- No official API
- CSV/XLS based integration
- Offline-first system
- [**CONSTRAINT**] All finalised accounting vouchers are immutable. Corrections must be made via reversal/void vouchers only.
- [**CONSTRAINT**] System architecture must be a modular monolith – no microservices.
- [**CONSTRAINT**] Inventory valuation method is Specific Identification exclusively. FIFO and LIFO are not applicable.
- [**CONSTRAINT**] Monetary values shall be stored and computed as DECIMAL(15,2). Floating-point types (FLOAT, DOUBLE) are prohibited for financial data.
- [**CONSTRAINT**] IMEI numbers must be exactly 15 decimal digits conforming to the GSMA standard.
- [**CONSTRAINT**] Maximum supported import file size is 10 MB per batch.
- [**CONSTRAINT**] Single-distributor in Version 2.0. Multi-company support is a future enhancement.

2.6 Assumptions

- Data formats remain consistent
- Tally is accessible locally
- The Lava portal CSV/XLS export format remains consistent with the documented schema (v1.3). If the format changes, field mappings must be reconfigured by an Admin.
- The distributor operates under Indian accounting standards and GST regulations.
- Bank statements are exported in XLS format from the bank's internet banking portal.
- At least one Admin user account is created during initial system setup via the onboarding wizard.
- The financial year follows the Indian financial year: April 1 to March 31.

3 External Interface Requirements

3.1 Use Case Model

Figure 7 illustrates the primary interactions between users and the system.

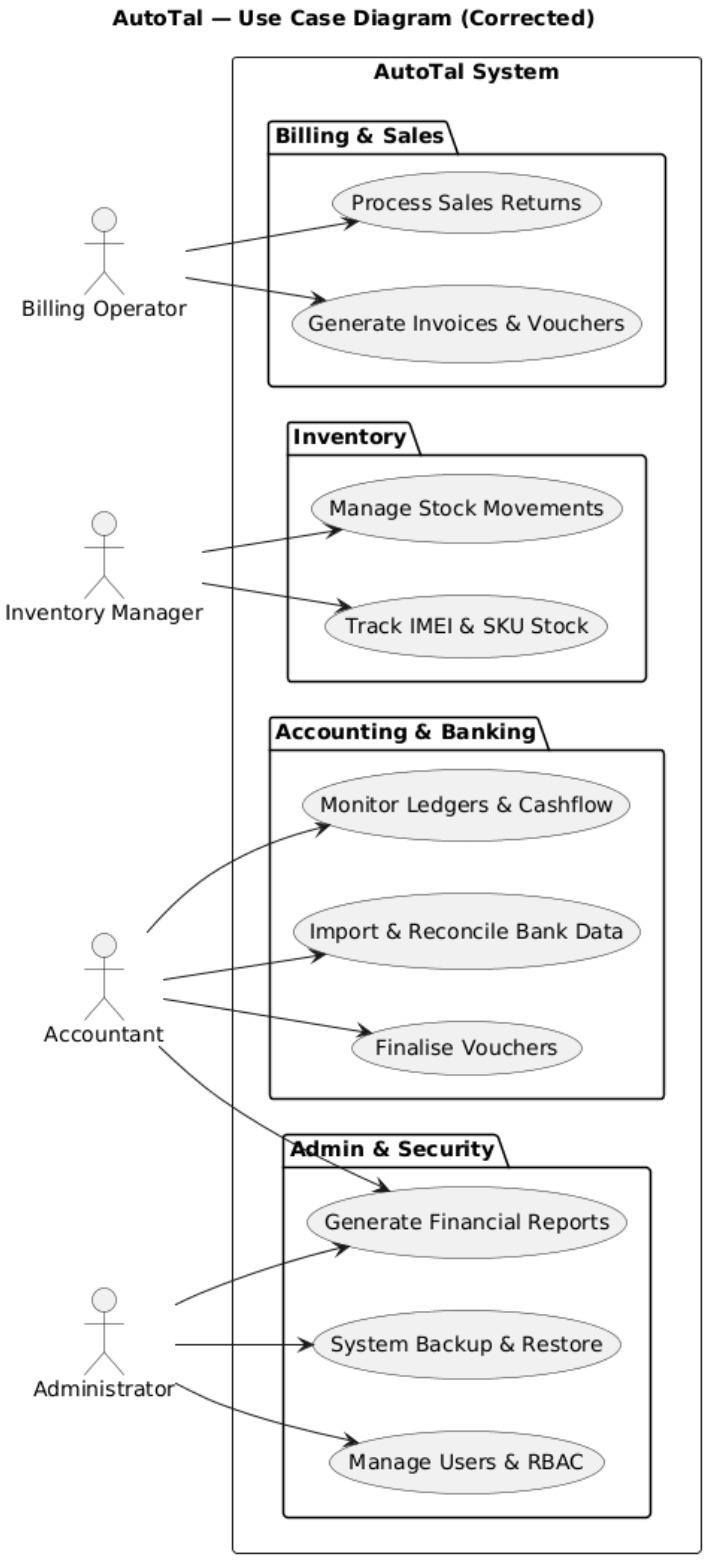


Figure 7: Use Case Diagram for AutoTal

3.2 User Interfaces

- Dashboard with summary (sales, stock, alerts) – refreshes within 2 seconds; shows MTD/YTD sales, current stock count, unresolved alert count, and last import timestamp.
- Import screen with validation preview – includes file upload control, configurable field mapping panel, row-by-row validation table (VALID / INVALID / WARNING per row), and confirm/cancel controls.
- Review screen before Tally sync – lists all Draft vouchers sortable by date, amount, and retailer; each expandable to show all constituent transactions.
- Reports section – date-range selectors and PDF/XLS export controls for Stock Statement, P&L, Balance Sheet, Retailer Ledger, and Bank Reconciliation Report.
- Alert Centre – lists unresolved alerts filterable by severity (INFO, WARN, ERROR); Resolve button per alert with mandatory resolution note.
- All destructive or irreversible actions (Finalise, Void, Delete batch, Restore backup) shall display a confirmation dialog before execution.
- Field-level validation errors shall be displayed inline, adjacent to the offending field.
- Minimum supported screen resolution: 1280×720 pixels.

3.3 Software Interfaces

- Tally ERP (voucher creation)
- File system (CSV/XLS)
- Database: PostgreSQL 15 via SQLAlchemy ORM; SQLite 3.40+ as fallback. Connection string configurable via encrypted settings file.
- Mobile/Web REST API: read-only endpoint at `http://localhost:8080/api/v1/` with JWT Bearer token authentication. Endpoints: `/dashboard`, `/reports/pnl`, `/reports/stock`, `/vouchers`, `/inventory`.
- PDF/XLS Export: ReportLab (PDF) and openpyxl (XLS). No external office suite required at runtime.
- Cloud Backup (optional): Google Drive API v3 (OAuth 2.0) for encrypted backup upload. Non-critical – failure does not impact core system operation.

4 System Features

4.1 Data Import Engine

4.1.1 Description

Imports and validates CSV/XLS data. Priority: High.

The Data Import Engine parses CSV and XLS/XLSX files exported from the Lava distributor portal and bank portals, validates their contents against a defined schema, and stages valid rows for Voucher Engine processing. Invalid rows are flagged with specific error codes and presented for user review before any data is committed to the database.

4.1.2 Stimulus/Response

User uploads file → System validates → Displays preview or error.

Stimulus (User Action)	**System Response**
User uploads a valid CSV/XLS file	System parses, maps fields, validates all rows, and displays preview table within 5 seconds
User uploads file exceeding 10 MB	System rejects immediately with ERR-IMP-001; does not begin parsing
User uploads unsupported file extension	System rejects with ERR-IMP-002
User confirms import with warnings present	System stages only valid rows, logs warnings, creates batch record, proceeds to Voucher Engine
User cancels import after preview	System discards all staged rows, deletes batch record, returns to import screen
File contains 0 valid rows	System halts import with ERR-IMP-003 and logs the failed batch
Duplicate reference number detected	System flags row as WARNING (WARN-IMP-001); user chooses skip or force reimport

4.1.3 Functional Requirements

- REQ-IMP-1: System shall accept CSV and XLS files up to 10MB. Accepted formats: CSV (.csv), Excel 97-2003 (.xls), and Excel Open XML (.xlsx). Any other extension triggers ERR-IMP-002 and the file is rejected without parsing.
- REQ-IMP-2: System shall validate mandatory fields (date, IMEI, amount). Specifically:
  - **Date:** Must conform to ISO 8601 (YYYY-MM-DD). Future dates are flagged as WARNING.
  - **IMEI Number:** Must be exactly 15 decimal digits; Luhn algorithm checksum (ISO/IEC 7812) applied.
  - **Amount:** Must be a positive DECIMAL(12,2). Zero or negative amounts are INVALID.
  - **Reference Number:** Non-empty, maximum 50 characters, alphanumeric with hyphens permitted.
  - **Retailer Name/Code:** Must match a retailer in the system or be flagged for new-retailer creation.
  - **Product Model:** Non-empty, maximum 100 characters.

- REQ-IMP-3: System shall reject files with invalid schema. A row missing any mandatory field is marked INVALID and excluded from staging. A human-readable error message with field name and rule violated is shown per invalid row.

- REQ-IMP-4: System shall log import errors with reason. Every import attempt – successful or failed – is logged with: batch ID, file name, file size, import type, timestamp (ISO 8601), user ID, total row count, valid/invalid/warning row counts. Persisted to the IMPORT_BATCH table.

- REQ-IMP-5: System shall prevent duplicate record imports. A record is a duplicate if: (a) the Reference Number already exists in the VOUCHER table, OR (b) the IMEI Number already exists in INVENTORY_IMEI with status IN_STOCK or SOLD. Duplicate rows are marked WARNING; the user chooses skip or force reimport per flagged row.

- REQ-IMP-6: System shall provide a configurable Field Mapping interface. For each import type (LAVA_SALES, LAVA_PURCHASE, BANK_STATEMENT), the Admin maps source column names to system schema fields. Configurations are saved per import type and loaded automatically on subsequent imports.

- REQ-IMP-7: System shall display a validation preview table before any data is staged. The table shows row number, status (VALID / INVALID / WARNING), and for INVALID/WARNING rows a human-readable error message.

- REQ-IMP-8: System shall complete parsing and validation of a 10 MB file containing up to 5,000 rows within 10 seconds on minimum hardware (4 GB RAM, dual-core 2 GHz CPU).

- REQ-IMP-9: System shall support rollback of an import batch. An Admin may cancel a committed import batch (before any vouchers are finalised) and revert all staged data atomically.

- REQ-IMP-10: System shall retain a 30-day history of import batches. Batches older than 30 days are archived to a compressed JSON Lines log file and removed from the active IMPORT_BATCH table.

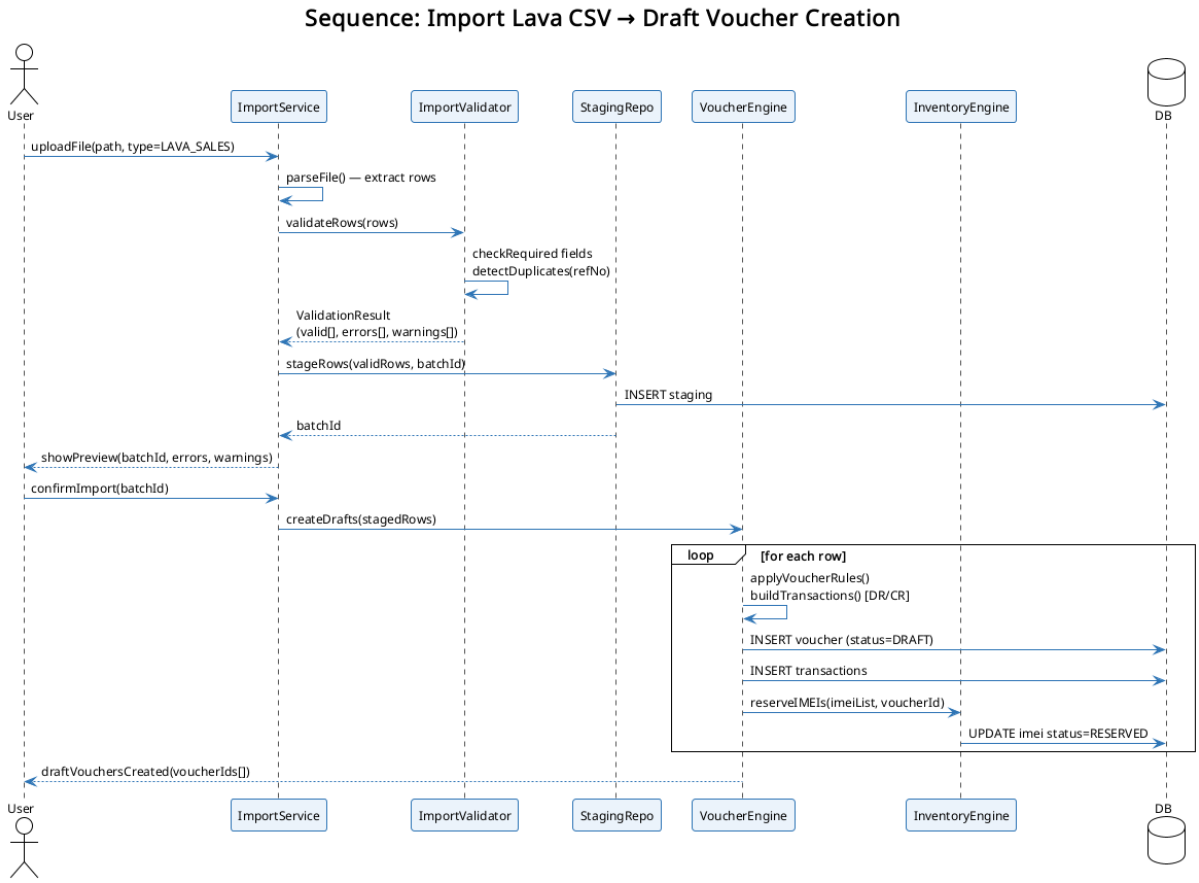


Figure 8: Import Process Sequence

Sequence Flow

4.2 Tally Synchronization

4.2.1 Description

Syncs validated data to Tally. Priority: High.

The Voucher Engine implements core double-entry accounting. It creates Draft vouchers from staged import data or manual entry, enforces debit/credit balance before finalisation, manages the lifecycle (DRAFT → FINAL → VOID), and ensures that finalised entries are immutable. Every financial posting is atomic.

4.2.2 Stimulus/Response

User confirms → System posts vouchers → Success/Failure shown.

Stimulus	Response
Import batch confirmed	System creates one Draft voucher per import row with all constituent DR/CR transactions

User finalises a Draft voucher	System verifies balance, posts ledger entries atomically, updates IMEI statuses, marks voucher FINAL, sets `finalised_at` timestamp
Voucher not balanced on finalise	System rejects finalisation; displays ERR-VCH-001 with DR and CR totals; raises ALERT
Admin voids a finalised voucher	System creates a mirror reversal voucher (negated amounts); marks original VOID; logs reason and user ID
User attempts to edit a finalised voucher	System rejects with ERR-VCH-002; instructs user to create a reversal voucher

4.2.3 Functional Requirements

- REQ-SYNC-1: System shall create sales vouchers. Each Sales Voucher contains at minimum one DEBIT to the Retailer Ledger and one CREDIT to the Sales Revenue Ledger. Voucher type: SALE.

- REQ-SYNC-2: System shall map retailer ledgers automatically. If a Retailer Ledger does not exist when creating a Sales Voucher, the system creates one automatically with name pattern `[RetailerName] Ledger` under the Sundry Debtors group.

- REQ-SYNC-3: System shall support review-before-post mode. The strict three-stage lifecycle is enforced: DRAFT (all fields editable) → FINAL (requires balance; immutable) → VOID (Admin-only; creates reversal entry). No voucher may skip or bypass stages.

- REQ-SYNC-4: System shall retry failed sync operations. The double-entry accounting rule is enforced at finalisation: $\sum \text{DEBIT} = \sum \text{CREDIT}$. If not met, finalisation is blocked and ERR-VCH-001 is raised.

- REQ-SYNC-5: System shall prevent duplicate voucher creation. A UNIQUE constraint is enforced on VOUCHER.reference_no. Duplicate reference attempts are rejected with ERR-VCH-003.

- REQ-SYNC-6: System shall rollback partial transactions on failure. Voucher finalisation executes as a single ACID-compliant database transaction. The following all succeed or all fail atomically: INSERT all ledger transaction entries; UPDATE IMEI statuses; UPDATE voucher status to FINAL and set `finalised_at`. On any failure, the entire transaction ROLLBACKS and the voucher remains DRAFT.

- REQ-SYNC-7: System shall create Purchase Vouchers from staged Lava Purchase data. Each Purchase Voucher contains at minimum one DEBIT to the Stock/Inventory Ledger and one CREDIT to the Lava/Supplier Payable Ledger. Voucher type: PURCHASE.

- REQ-SYNC-8: System shall support Journal Vouchers (type JOURNAL) for adjustments, corrections, and opening balance entries. Same lifecycle and balance enforcement applies.

- REQ-SYNC-9: System shall support Payment Vouchers (type PAYMENT) to record cash or bank payments received from retailers or made to Lava.
- REQ-SYNC-10: System shall retain full audit history for all vouchers. Finalised and voided vouchers store permanently: voucher ID, type, status, all transactions, finalising user, timestamp, void reason (if applicable), and reversal voucher reference (if applicable).

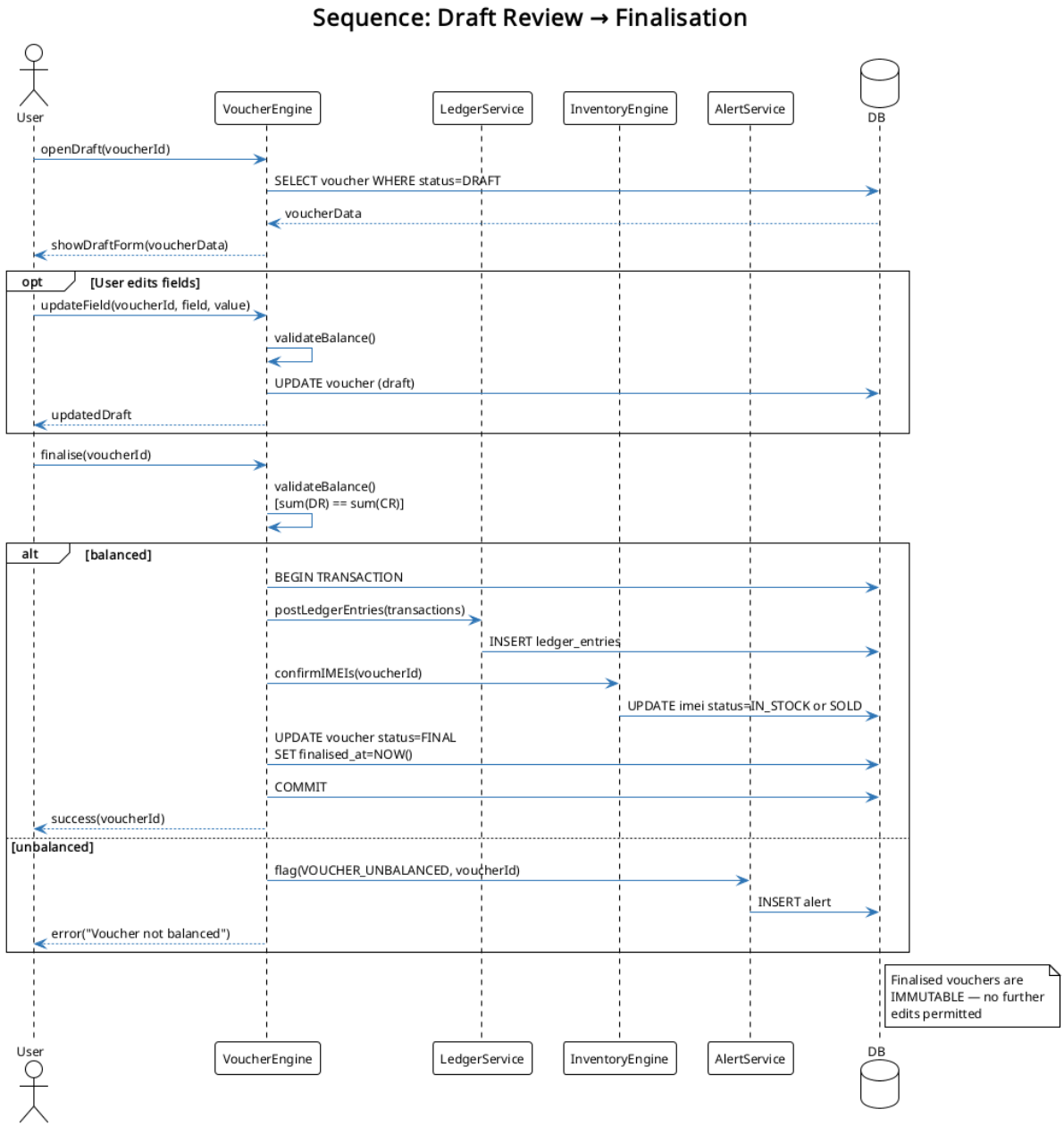


Figure 9: Voucher Finalisation Sequence

Voucher Finalisation Flow

4.3 Inventory Management

4.3.1 Description

Tracks IMEI-level stock. Priority: High.

The Inventory Management module tracks every mobile handset at the granularity of its IMEI number. Each IMEI carries its own purchase cost (Specific Identification method). The module manages the full lifecycle from receipt (Primary Stock) to sale (Secondary Sales), including stock adjustments and mismatch detection.

4.3.2 Stimulus/Response

Stock import → System updates inventory → Flags mismatch.

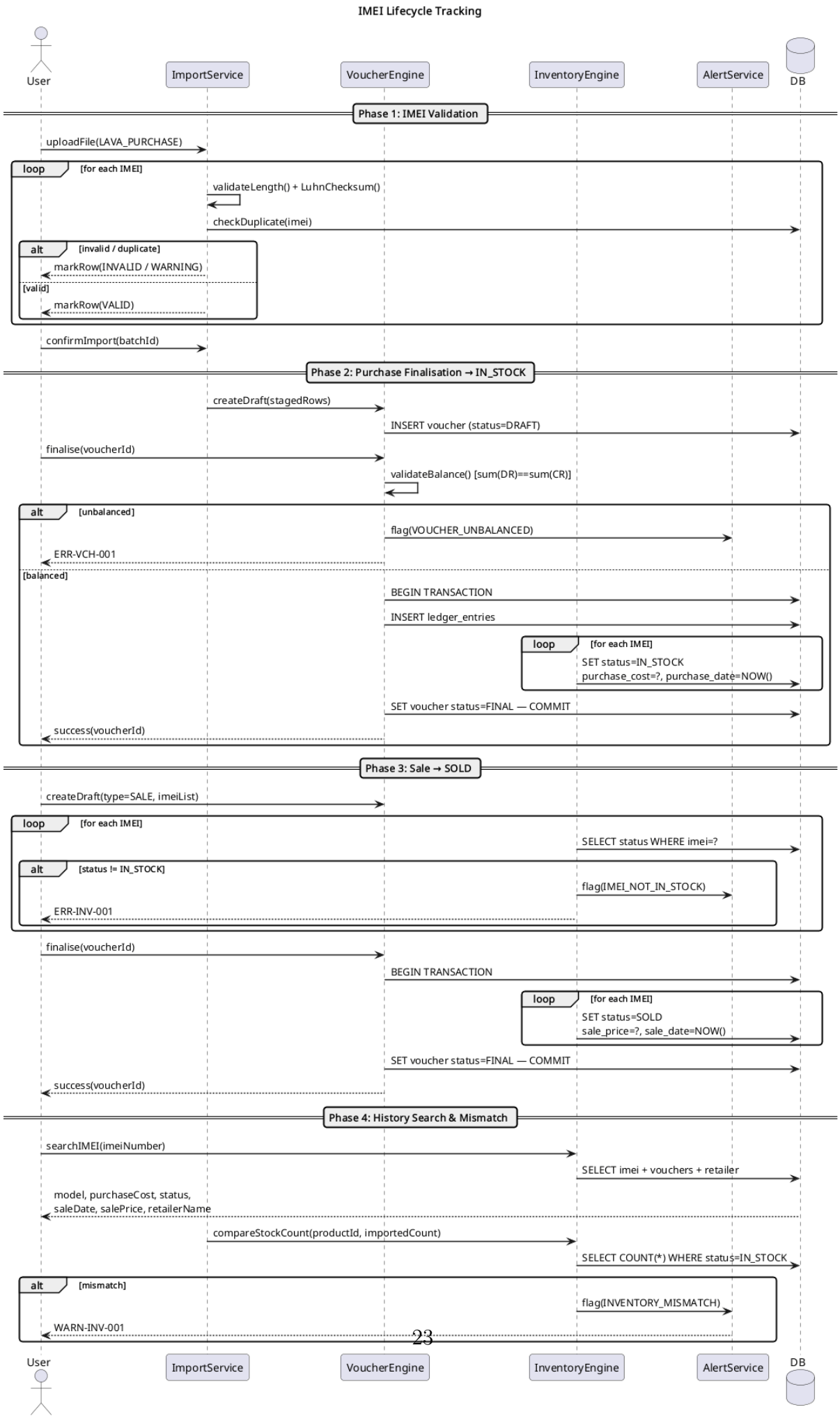
Stimulus	**Response**
Purchase voucher finalised with IMEI list	System creates INVENTORY_IMEI records for each IMEI (status: IN_STOCK, purchase_cost set)
Sale voucher finalised with IMEI list	System updates each IMEI to SOLD; sets sale_price, sale_date, sale_voucher_id
IMEI already SOLD listed in new sale	System raises ERR-INV-001; finalisation blocked
IMEI not found in system	System raises ERR-INV-002; finalisation blocked
Stock count in import differs from system	System raises WARN-INV-001 with system count vs import count

4.3.3 Functional Requirements

- REQ-INV-1: System shall store IMEI as a unique 15-digit identifier. Stored as UNIQUE VARCHAR(20) in INVENTORY_IMEI. UNIQUE INDEX enforced at database level. Duplicate IMEI insertion rejected with ERR-INV-003.
- REQ-INV-2: System shall prevent duplicate IMEI entries. Every IMEI is validated against: (a) length exactly 15 characters; (b) decimal digits only (0–9); (c) Luhn algorithm checksum (ISO/IEC 7812). IMEIs failing any rule are marked INVALID at import.
- REQ-INV-3: System shall classify stock as primary or secondary.
  - **Primary Stock:** Units received from Lava via a Purchase Voucher. `stock_type = PRIMARY`.
  - **Secondary Stock:** Units returned by retailers via a Return/Credit Voucher. `stock_type = SECONDARY`.
- REQ-INV-4: System shall track stock movement per IMEI. The Specific Identification valuation method is applied: each IMEI record stores its individual purchase_cost (DECIMAL(12,2)) set at purchase voucher finalisation. This cost is immutable once the purchase voucher is finalised. Stock valuation = SUM(purchase_cost) of all IN_STOCK IMEIs. Valid IMEI lifecycle statuses: IN_STOCK → SOLD;

IN_STOCK → ADJUSTED; ADJUSTED → IN_STOCK. No other transitions are valid.

- REQ-INV-5: System shall flag inventory mismatches. When the system's IN_STOCK count for a product differs from the count reported in an imported Lava stock statement, WARN-INV-001 is raised with system count vs import count.
- REQ-INV-6: System shall provide full IMEI history search. Given any IMEI number, the system returns: product model, brand, purchase date, purchase cost, purchase voucher ID, current status, sale date (nullable), sale price (nullable), sale voucher ID (nullable), retailer name (nullable), and any associated alerts.
- REQ-INV-7: System shall support bulk IMEI operations. A Purchase Voucher may contain up to 500 IMEI numbers, all inserted via a single batch database operation within 3 seconds.


IMEI Lifecycle Tracking
User
ImportService
VoucherEngine
InventoryEngine
AlertService
DB
Phase 1: IMEI Validation
uploadFile(LAVA_PURCHASE)
loop
[for each IMEI]
validateLength() + LuhnChecksum()
checkDuplicate(imei)
alt
[invalid / duplicate]
markRow(INVALID / WARNING)
[valid]
markRow(VALID)
confirmImport(batchId)
Phase 2: Purchase Finalisation → IN_STOCK
createDraft(stagedRows)
INSERT voucher (status=DRAFT)
finalise(voucherId)
validateBalance() [sum(DR)==sum(CR)]
alt
[unbalanced]
flag(VOUCHER_UNBALANCED)
ERR-VCH-001
[balanced]
BEGIN TRANSACTION
INSERT ledger_entries
loop
[for each IMEI]
SET status=IN_STOCK
purchase_cost=?, purchase_date=NOW()
SET voucher status=FINAL — COMMIT
success(voucherId)
Phase 3: Sale → SOLD
createDraft(type=SALE, imeiList)
loop
[for each IMEI]
SELECT status WHERE imei=?
alt
[status != IN_STOCK]
flag(IMEI_NOT_IN_STOCK)
ERR-INV-001
finalise(voucherId)
BEGIN TRANSACTION
loop
[for each IMEI]
SET status=SOLD
sale_price=?, sale_date=NOW()
SET voucher status=FINAL — COMMIT
success(voucherId)
Phase 4: History Search & Mismatch
searchIMEI(imeiNumber)
SELECT imei + vouchers + retailer
model, purchaseCost, status,
saleDate, salePrice, retailerName
compareStockCount(productId, importedCount)
SELECT COUNT(*) WHERE status=IN_STOCK
alt
[mismatch]
flag(INVENTORY_MISMATCH)
WARN-INV-001

4.4 Retailer Management

4.4.1 Functional Requirements

- REQ-RET-1: System shall maintain ledger per retailer. A dedicated Ledger account is automatically created under the Sundry Debtors group when a new retailer is added. Deletion of a Retailer Ledger with non-zero balance is blocked.

- REQ-RET-2: System shall track credit balance. Outstanding balance = (sum of sales) − (sum of payments received). Recomputed after every finalised Sale Voucher or Payment Voucher. If projected post-sale outstanding exceeds the retailer's credit_limit, WARN-RET-001 is raised (non-blocking – the user may still proceed).

- REQ-RET-3: System shall display transaction history. Per-retailer history shows: date, voucher ID, type, amount (DR/CR), running balance; sorted by date descending; filterable by date range; exportable as PDF and XLS.

- REQ-RET-4: System shall store retailer master data: Name (VARCHAR(100), NOT NULL), Phone (VARCHAR(15), Indian mobile pattern `^[6-9][0-9]{9}$`), Address (TEXT, nullable), GSTIN (VARCHAR(15), standard GST format, nullable), Credit Limit (DECIMAL(12,2), default 0), Created At (TIMESTAMP, auto-set).

- REQ-RET-5: System shall prevent hard deletion of a Retailer record if any finalised vouchers reference it. The retailer is instead marked INACTIVE and hidden from new voucher dropdowns but visible in historical reports.

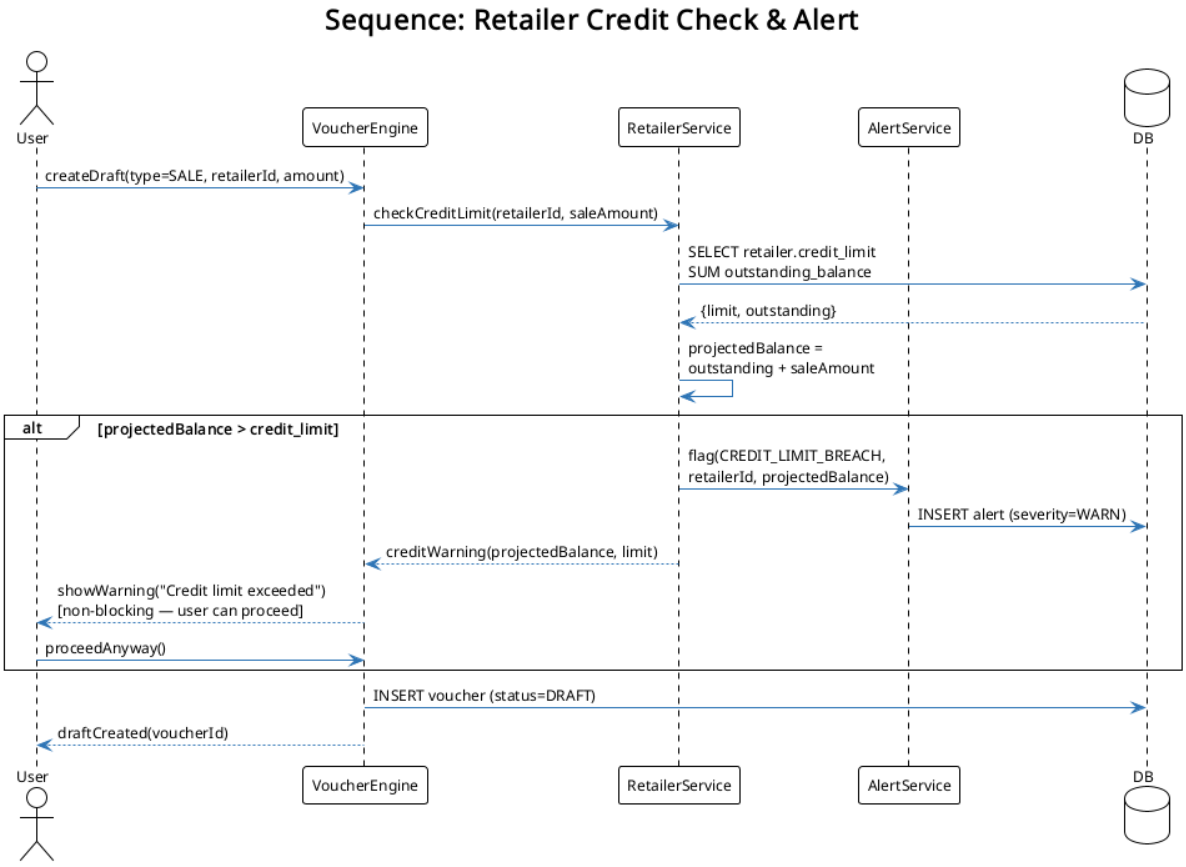


Figure 11: Credit Limit Alert Sequence

Credit Limit Alert Flow

4.5 Bank Reconciliation

4.5.1 Functional Requirements

- REQ-BANK-1: System shall import bank statements (XLS). Extracted fields per row: transaction date, value date (nullable), description (VARCHAR(255)), debit (nullable), credit (nullable), balance (nullable). Rows with both debit and credit null are skipped with a WARNING log entry.

- REQ-BANK-2: System shall match transactions by date and amount. Auto-matching criteria (in priority order): (1) Exact amount AND transaction date within ±1 calendar day; (2) Exact amount AND description contains the voucher reference number. A match is confirmed only if exactly one ledger transaction satisfies the criteria; multiple candidates are flagged for manual matching.

- REQ-BANK-3: System shall flag unmatched entries. Three reconciliation statuses: UNMATCHED (default), MATCHED (linked to ledger transaction), IGNORED (user-excluded). Transitions: UNMATCHED → MATCHED, UNMATCHED → IGNORED, MATCHED → UNMATCHED.

- REQ-BANK-4: System shall generate a Bank Reconciliation Statement showing: balance per bank statement, deposits in transit, outstanding cheques, balance per

books, and all UNMATCHED transactions with date/description/amount. Exportable as PDF and XLS.

- REQ-BANK-5: System shall raise an INFO alert for every bank transaction remaining UNMATCHED for more than 30 calendar days from its transaction date.

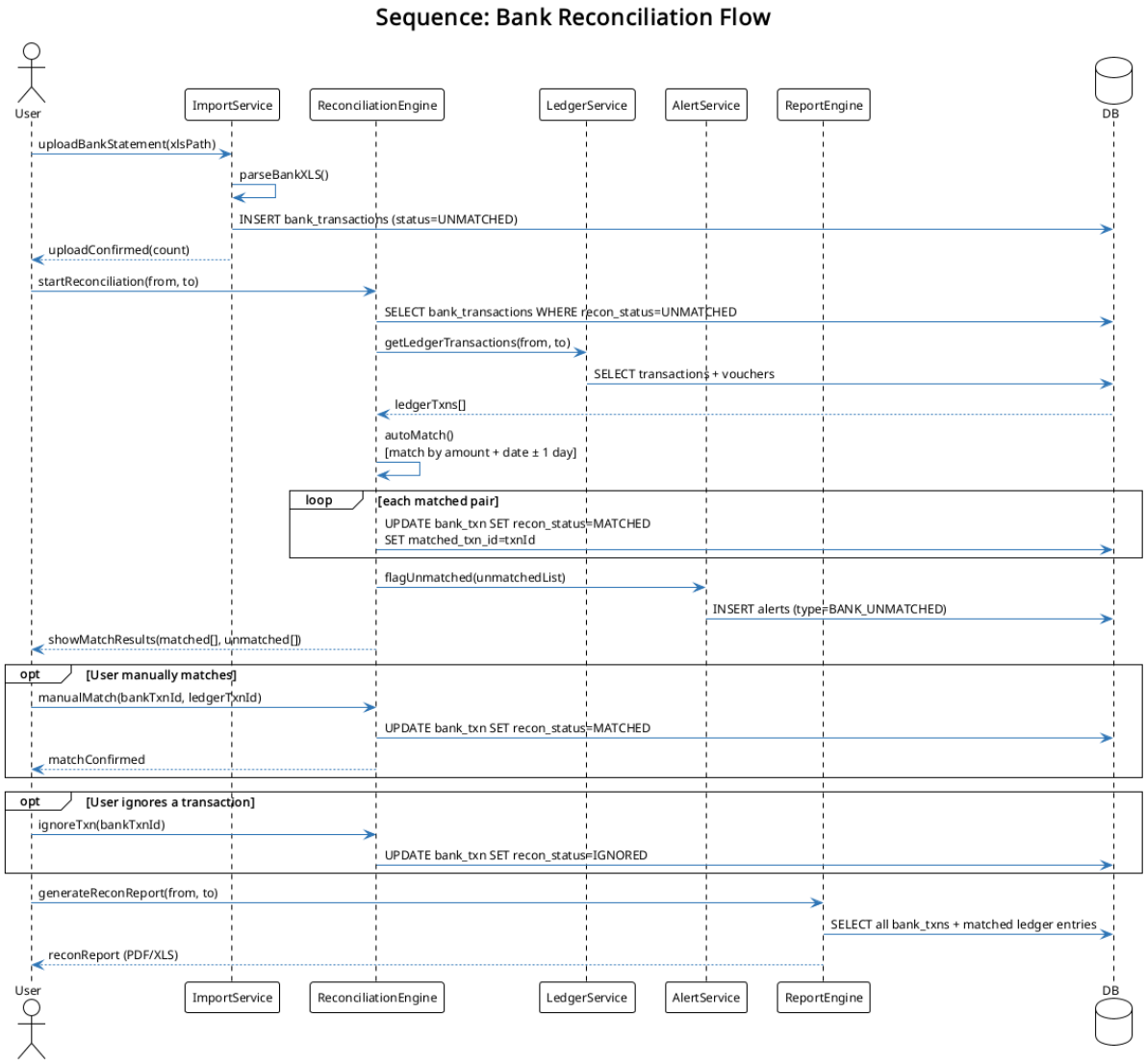


Figure 12: Bank Reconciliation Sequence

Reconciliation Flow

4.6 Stock Statement Generator

4.6.1 Functional Requirements

- REQ-REP-1: System shall generate stock statements by date range. Per product model: Opening Stock (units and value), Receipts, Sales, Closing Stock. Values computed using Specific Identification: SUM(purchase_cost) of IN_STOCK IMEIs.

- REQ-REP-2: System shall auto-fill calculated values. All stock figures auto-computed from INVENTORY_IMEI using the formula: Closing Stock = Opening Stock + Receipts − Sales ± Adjustments. Any discrepancy between the formula result and actual IN_STOCK count is flagged as ERR-REP-001.
- REQ-REP-3: System shall allow manual overrides. All overrides logged with: user ID, timestamp, original value, overridden value, and mandatory reason (minimum 10 characters). Overrides highlighted in the exported report.
- REQ-REP-4: System shall save report templates. Named configurations specify: date range type, product filter, and column selection. Templates are saved and reloaded for subsequent report generation.
- REQ-REP-5: System shall generate the following additional financial reports, all exportable as PDF and XLS within 10 seconds for a one-year date range:
  - **P&L Statement:** Revenue, COGS (Specific Identification), Gross Profit, Operating Expenses, Net Profit.
  - **Balance Sheet:** Assets (Stock, Bank, Cash, Debtors), Liabilities (Payables), Equity.
  - **Trial Balance:** All ledger accounts with opening balance, total DR, total CR, closing balance.
  - **Retailer Ledger:** Full transaction history per retailer for any date range.

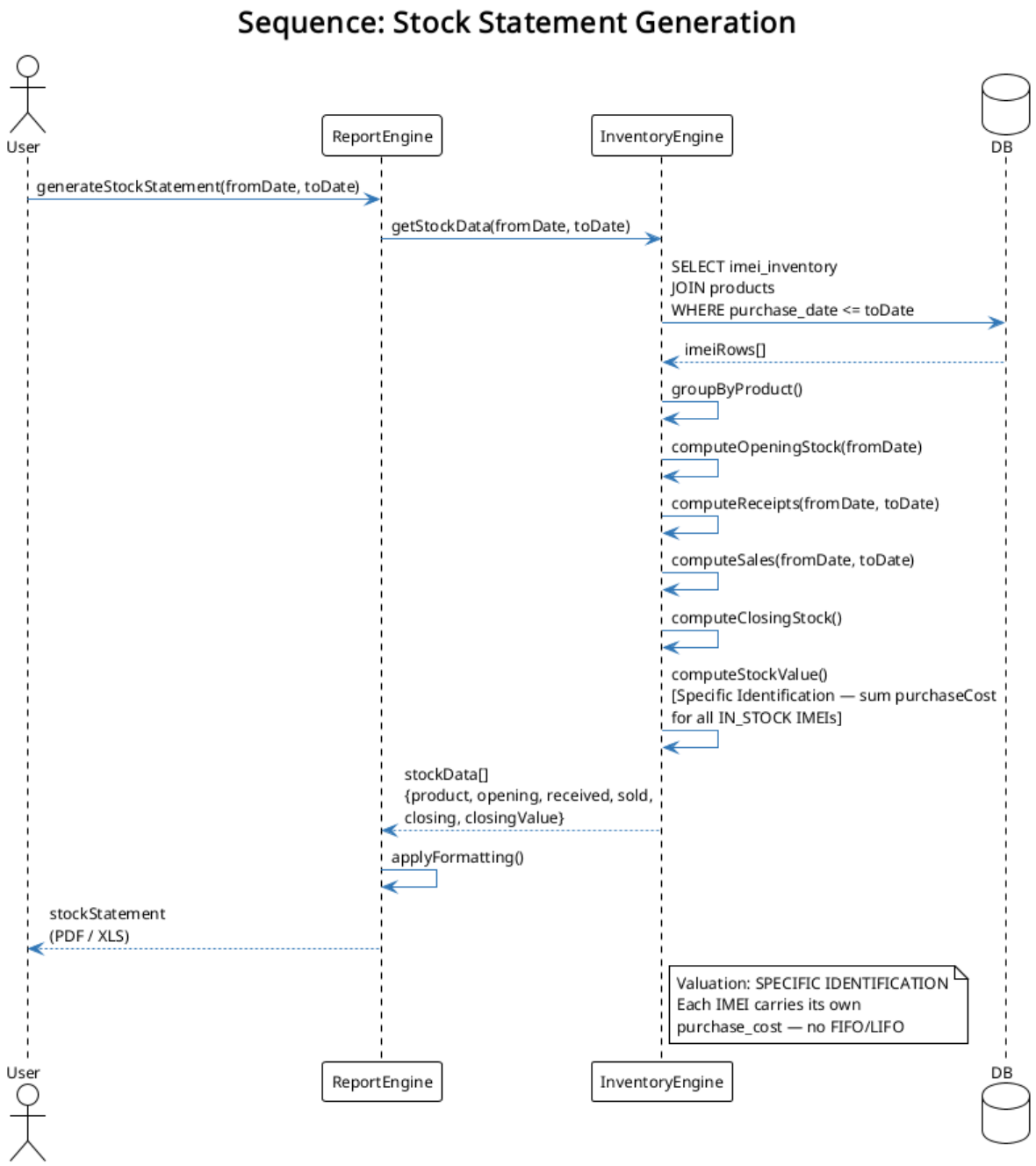


Figure 13: Stock Statement Generation Sequence

Stock Statement Generation Flow

4.7 Pricing Module

- REQ-PRICE-1: System shall import price list. Price list stored per product model with fields: product_id (FK), effective_from (DATE), effective_to (DATE, nullable), mrp (DECIMAL(10,2)), distributor_price (DECIMAL(10,2)), created_by (FK to USER). Importable from CSV with schema: model_name, effective_from, mrp, distributor_price.

- REQ-PRICE-2: System shall allow manual price overrides. Overrides record: orig-

inal price, override amount, user, and mandatory reason (minimum 5 characters). Overrides exceeding 15% deviation from the current price list require Admin-level approval.

- REQ-PRICE-3: System shall maintain price history. No price record is deleted. Superseded prices are marked INACTIVE but retained in the database.

4.8 Alert System

- REQ-ALERT-1: System shall flag mismatches. The alert system is non-blocking: anomaly detection flags issues in the ALERT table without preventing the user's primary workflow. Alert Centre badge count updates within 5 seconds of alert creation.

- REQ-ALERT-2: System shall categorize alerts (Low/Medium/High). Three severity levels:
  - **ERROR:** Data integrity violation requiring immediate resolution (e.g., IMEI already SOLD, voucher unbalanced).
  - **WARN:** Business rule advisory requiring review (e.g., credit limit exceeded, duplicate reference).
  - **INFO:** Informational notification (e.g., bank transaction unmatched for 30 days).

- REQ-ALERT-3: System shall generate alerts for the following specific conditions:

Alert Type	**Severity**	**Trigger Condition**
IMEI_DUPLICATE	ERROR	IMEI already exists in INVENTORY_IMEI
IMEI_NOT_IN_STOCK	ERROR	Sale voucher IMEI status $\neq$ IN_STOCK
VOUCHER_UNBALANCED	ERROR	sum(DR) $\neq$ sum(CR) on finalise
STOCK_NEGATIVE	ERROR	Closing stock count $<$ 0 for any product
CREDIT_LIMIT_BREACH	WARN	Retailer projected outstanding $>$ credit_limit
DUPLICATE_IMPORT	WARN	reference_no already exists in VOUCHER
INVENTORY_MISMATCH	WARN	System IN_STOCK $\neq$ import-reported count
PRICE_OVERRIDE_EXCESS	WARN	Price override $>$ 15% deviation from price list
BANK_UNMATCHED_30D	INFO	Bank transaction UNMATCHED $>$ 30 days

- REQ-ALERT-4: Each alert record shall store: alert_id (PK), alert_type (VARCHAR(50)), severity (ENUM: INFO/WARN/ERROR), message (TEXT with actual values), entity_type (VARCHAR(30)), entity_id (VARCHAR(50)), resolved (BOOLEAN, default false), created_at (TIMESTAMP), resolved_at (nullable), resolved_by (FK to USER, nullable).

- REQ-ALERT-5: System shall allow users to resolve alerts via a Resolve button. Resolution requires a mandatory note (minimum 10 characters). Resolved alerts are retained indefinitely and visible in an Archived Alerts view.

4.9 Backup System

- REQ-BACKUP-1: System shall perform automatic daily backups at a user-configured time (default: 02:00 IST), including the complete database and application configuration file. PostgreSQL: `pg_dump` custom format. SQLite: atomic file copy.
- REQ-BACKUP-2: System shall allow manual backup triggered by Admin at any time. Naming convention: `autotel_backup_YYYYMMDD_HHMMSS.bak.enc`. All backup files are encrypted using AES-256 (key derived via PBKDF2, SHA-256, minimum 100,000 iterations from a user-configured passphrase). Unencrypted backup files are not retained on disk after encryption.
- REQ-BACKUP-3: System shall retain 30 daily backups locally before overwriting the oldest. Monthly backups (first of each calendar month) retained for 12 months. Admin notified via Alert Centre on scheduled backup failure.
- REQ-BACKUP-4: System shall support one-click restore from any retained backup. Before restore: (a) display backup timestamp and record count; (b) create a safety backup of current database; (c) require explicit user confirmation. Restore operation is atomic.
- REQ-BACKUP-5: System shall optionally upload completed backup files to a configured Google Drive folder (OAuth 2.0). Upload occurs in the background without blocking desktop operations. Upload failure generates INFO alert but does not affect local backup completion status.

4.10 Access System

- REQ-ACC-1: System shall support single-user authentication. Role-based access control (RBAC) with two roles: ADMIN (full access including configuration, void vouchers, backup/restore, user management) and ACCOUNTANT (operational access: import, review, finalise, reports, bank reconciliation; cannot void vouchers, manage users, configure mappings, or restore backups). Passwords hashed with bcrypt (minimum cost factor 12). Session duration maximum 8 hours; terminated on logout or timeout.
- REQ-ACC-2: System shall provide read-only mobile access via a REST API (FastAPI or Flask) exposing read-only endpoints with JWT Bearer token authentication (maximum 24-hour expiry). No write or create endpoints on the mobile API. System shall lock a user account after 5 consecutive failed login attempts within a 15-minute window; locked accounts require Admin intervention to unlock.
- REQ-ACC-3: System shall maintain a login audit log recording: user ID, username, login timestamp, logout/timeout timestamp, and login result (SUCCESS/-FAILED). This log is non-deletable by any user role.

5 Other Nonfunctional Requirements

5.1 Performance

- System shall process 10MB files within 5 seconds.

- System shall support 1000+ records per import.
- System shall complete parsing and validation of a 10 MB / 5,000-row file within 10 seconds on minimum hardware.
- Single voucher finalisation shall complete within 2 seconds including all database writes.
- Bulk IMEI insert of 500 records shall complete within 3 seconds.
- Dashboard shall load within 2 seconds of navigation.
- All report types (P&L, Balance Sheet, Stock Statement) shall generate within 10 seconds for a one-year date range.
- IMEI search shall return a result within 1 second.
- Bank auto-match of 1,000 transactions shall complete within 15 seconds.
- Full daily backup of a 1 GB database shall complete within 5 minutes.

5.2 Reliability

- System shall ensure atomic transactions.
- System shall recover from failure without data loss.
- System shall ensure zero data loss for all finalised vouchers through ACID-compliant database transactions and WAL (Write-Ahead Logging).
- System shall recover from an unexpected application crash and resume normal operation within 30 seconds of restart, without manual database intervention.
- System shall detect and automatically rollback incomplete transactions caused by power failure or crash on the next startup, using PostgreSQL WAL recovery or SQLite journal mode.
- Scheduled backup failure rate shall remain below 1%. Backup failures shall generate an ERROR alert within 5 minutes of the scheduled backup time.
- System shall maintain 99% uptime during business hours (09:00–18:00 IST, Monday–Saturday), excluding planned maintenance windows.

5.3 Security

- Session-based authentication
- Local data encryption
- User passwords shall be hashed using bcrypt with a minimum cost factor of 12. Plaintext passwords are never stored or logged.
- The local database directory shall be accessible only to the application process and the OS admin user.

- All backup files shall be encrypted with AES-256 before writing to disk.
- The mobile REST API shall require JWT Bearer token authentication on every request; tokens expire after a maximum of 24 hours.
- The application configuration file (containing database connection strings) shall be stored encrypted using the OS keychain or an equivalent secure storage mechanism.
- IMEIs in application logs shall be masked as `XXXXXXXXXXXXX999` (last 3 digits visible only).
- User accounts shall be locked after 5 consecutive failed login attempts (see REQ-ACC-2).

5.4 Usability

Simple UI for non-technical users.

- A new user with no prior accounting experience shall complete a full import-review-finalise cycle within 15 minutes after the onboarding tutorial.
- Every financial field shall have a tooltip explaining its purpose and acceptable format.
- All error messages shall be in plain English and include a suggested next action.
- Keyboard shortcuts shall be provided for: Import (Ctrl+I), Review Drafts (Ctrl+R), Generate Report (Ctrl+G).
- A progress indicator shall be displayed for any operation exceeding 2 seconds, showing percentage completion where calculable or an indeterminate spinner otherwise.

5.5 Maintainability

Modular architecture.

- The system shall be structured as a modular monolith with clearly separated modules (Import, VoucherEngine, InventoryEngine, LedgerService, ReconciliationEngine, ReportEngine, AlertService, DAL). Inter-module communication via defined service interfaces only.
- The codebase shall maintain a minimum 80% unit test coverage for all service layer classes.
- Field mapping configuration shall be data-driven (stored in the database, not hard-coded) to accommodate Lava portal format changes without code changes.
- Database schema changes shall be managed via versioned migration scripts (e.g., Alembic for Python). No manual SQL alterations required for version upgrades.
- All application logging shall use structured JSON Lines format for automated log analysis.

5.6 Scalability

- System shall operate without performance degradation up to 500,000 IMEI records, ensured by database indexes on imei_number, status, product_id, and purchase_date.
- Migration from SQLite to PostgreSQL shall require only a connection string change – no business logic changes.
- The REST API layer shall be stateless (no server-side session state in application memory) to support horizontal scaling in a future multi-user version.

6 Business Rules

- All entries must be verified before posting.
- IMEI must be unique.
- All finalised vouchers are permanent records. Corrections shall be made exclusively by creating a void (reversal) voucher and, if needed, a new corrected voucher.
- Every financial posting shall create at least one DEBIT and one CREDIT of equal total amount. Enforced at the service layer before any database write.
- Each IMEI number represents a unique physical device and appears exactly once in the system across its lifecycle.
- Inventory valuation uses Specific Identification exclusively: COGS for any sale = the exact purchase_cost of the sold IMEI(s).
- Credit limit checks are non-blocking. The system warns but does not prevent sales to a retailer whose credit limit is exceeded.
- The Indian Financial Year (April 1 – March 31) is the default period for reports, trial balance, and opening balance computation. Configurable by Admin during setup.
- Manual price overrides exceeding 15% deviation from the current price list require Admin-level role to apply.
- Finalised vouchers, finalised IMEI records, and resolved alerts shall never be permanently deleted from the database.
- No import data shall be committed to the Ledger or Inventory without passing through the Draft Voucher stage and explicit user finalisation. Automated finalisation without user review is prohibited.
- Unmatched bank transactions shall not block any accounting or inventory operation. Bank reconciliation is a reporting and audit activity only.

7 Error Handling Requirements

- REQ-ERR-1: System shall log every application error with: error_code (VARCHAR(20)), module (VARCHAR(30)), timestamp (ISO 8601), user_id (nullable), entity_type (nullable), entity_id (nullable), error_message (TEXT), stack_trace (TEXT). Logs written to rotating `autotel.log` with daily rotation and 30-day retention.
- REQ-ERR-2: System shall rollback the entire database transaction on any error during a multi-step atomic operation (voucher finalisation, import commit, backup restore). Partial commits are strictly prohibited.
- REQ-ERR-3: System shall display user-friendly error messages for all recoverable errors. Messages shall: (a) state what went wrong in plain language; (b) include the error code for support reference; (c) suggest a corrective action. Stack traces are not displayed to end users.
- REQ-ERR-4: System shall handle the following edge cases without crashing:
  - Empty CSV/XLS file (0 rows after header): display ERR-IMP-003; abort gracefully.
  - CSV file with missing header row: display ERR-IMP-004; abort.
  - Database connection failure during import commit: rollback staged data; display ERR-DB-001; retain file for re-import.
  - Disk full during backup: raise ERROR alert; log failure; do not corrupt the existing backup.
  - Power loss during voucher finalisation: on next startup, detect incomplete transactions via PostgreSQL WAL and rollback automatically.
- REQ-ERR-5: System shall provide an error history view in the Admin panel showing the last 500 application errors, searchable by date range, module, and error code.

8 Data Integrity Requirements

- REQ-INT-1: System shall enforce referential integrity via database-level foreign key constraints for all inter-table relationships. Cascading deletes are prohibited – all deletes are soft deletes (marking INACTIVE) or blocked if referenced records exist.
- REQ-INT-2: System shall enforce the following database-level CHECK constraints:
  - TRANSACTION.amount > 0
  - VOUCHER.voucher_type IN ('PURCHASE', 'SALE', 'JOURNAL', 'PAYMENT')
  - VOUCHER.status IN ('DRAFT', 'FINAL', 'VOID')
  - INVENTORY_IMEI.status IN ('IN_STOCK', 'SOLD', 'ADJUSTED')
  - BANK_TRANSACTION.recon_status IN ('UNMATCHED', 'MATCHED', 'IGNORED')

– ALERT.severity IN ('INFO', 'WARN', 'ERROR')

- REQ-INT-3: System shall enforce IMEI uniqueness at the database level via a UNIQUE INDEX on INVENTORY_IMEI.imei_number. This constraint cannot be disabled by any application operation.
- REQ-INT-4: System shall enforce voucher immutability at the application (service) layer. Any attempt to UPDATE a FINAL voucher – except setting status to VOID – is blocked with ERR-VCH-002. This check is implemented in the service layer, not only in the UI.
- REQ-INT-5: All monetary computations shall use DECIMAL(15,2) arithmetic. Financial values shall never be converted to floating-point types. Rounding uses half-up rounding (ROUND_HALF_UP) to two decimal places.

9 Appendix A: Glossary

Term / Abbreviation	**Full Form**	**Definition**
ACID	Atomicity, Consistency, Isolation, Durability	Four properties guaranteeing reliable database transaction processing. Atomicity: all-or-nothing. Consistency: DB stays valid. Isolation: concurrent transactions do not interfere. Durability: committed data survives failures.
AES	Advanced Encryption Standard	Symmetric block cipher. AES-256 uses a 256-bit key. Used for backup file encryption.
Alembic	—	A lightweight database migration tool for Python / SQLAlchemy, used to version-control schema changes.
API	Application Programming Interface	A defined interface through which software components communicate.
Balance Sheet	—	A financial statement showing assets, liabilities, and equity at a specific date. Equation: Assets = Liabilities + Equity.

Term / Abbreviation	Full Form	Definition
bcrypt	—	A password hashing function based on the Blowfish cipher, designed to be computationally expensive to resist brute-force attacks. AutoTal uses minimum cost factor 12.
BCNF	Boyce-Codd Normal Form	Stronger version of 3NF. A relation is in BCNF if for every non-trivial dependency X→Y, X is a superkey. AutoTal's INVENTORY_IMEI has one documented BCNF deviation (sale_price stored for performance).
COGS	Cost of Goods Sold	Direct costs attributable to goods sold. In AutoTal: SUM(purchase_cost) of sold IMEIs (Specific Identification).
CR	Credit	In double-entry accounting, a credit increases liabilities, equity, or revenue, or decreases assets or expenses.
CSV	Comma-Separated Values	A plain-text file format where each line is a record and fields are separated by commas.
DAL	Data Access Layer	The software layer abstracting database operations (queries, inserts, updates) from business logic. Implemented as Repository classes in AutoTal.
DB	Database	An organised, structured collection of data stored and managed electronically.
DBMS	Database Management System	Software managing creation, maintenance, and querying of a database (e.g., PostgreSQL, SQLite).
DECIMAL(p,s)	—	A fixed-precision SQL numeric type. p = total digits, s = digits after decimal. Used for all financial values in AutoTal.

Term / Abbreviation	Full Form	Definition
DFD	Data Flow Diagram	A graphical representation of data flow through a system, showing processes, data stores, and external entities.
Draft Voucher	—	A voucher in DRAFT status. All fields are editable. Not yet posted to the ledger. No IMEI status changes are made.
DR	Debit	In double-entry accounting, a debit increases assets or expenses, or decreases liabilities, equity, or revenue.
ENUM	Enumeration	A data type with a fixed, finite set of named values (e.g., status: DRAFT, FINAL, VOID).
ER	Entity-Relationship	A data modelling technique showing entities (tables) and their relationships.
ERD	Entity-Relationship Diagram	A visual representation of the ER model used for database schema design.
ERP	Enterprise Resource Planning	Integrated management software for accounting, inventory, and business operations (e.g., Tally ERP).
FIFO	First-In, First-Out	Inventory costing method where first units received are first costed on sale. **Not used in AutoTal.**
FK	Foreign Key	A column referencing the Primary Key of another table, enforcing referential integrity.
GST	Goods and Services Tax	Indirect tax levied on supply of goods and services in India.
GSMA	Global System for Mobile Communications Association	Industry body governing the IMEI standard for global mobile device identification.

Term / Abbreviation	Full Form	Definition
GSTIN	Goods and Services Tax Identification Number	15-character alphanumeric identifier for GST-registered businesses in India. Format: 2-digit state code + 10-digit PAN + 1-digit entity number + 'Z' + 1 check digit.
HSN	Harmonized System of Nomenclature	A 4-to-8-digit code classifying traded products, used in Indian GST invoicing.
HTTP	HyperText Transfer Protocol	Application-layer protocol forming the foundation of web data communication.
HTTPS	HTTP Secure	HTTP encrypted via TLS.
IEEE	Institute of Electrical and Electronics Engineers	Professional association publishing technical standards. IEEE Std 830-1998 governs this SRS format.
IMEI	International Mobile Equipment Identity	A unique 15-digit decimal identifier for every mobile handset globally. Structure: TAC (8 digits) + SNR (6 digits) + CD (1 Luhn check digit). Governed by GSMA.
Ind AS	Indian Accounting Standards	Accounting standards adopted by ICAI, aligned with IFRS.
IndLava	—	The internal distributor-facing portal operated by Lava International for managing orders, stock, and sales data.
INR	Indian Rupee	Official currency of India. ISO 4217 code: INR.
IST	Indian Standard Time	UTC+5:30. The time zone used for all AutoTal timestamps.
ISO 8601	International Standard 8601	International standard for date and time representation. AutoTal uses YYYY-MM-DD for dates.
ISO/IEC 7812	International Standard 7812	Standard defining the Luhn algorithm used for IMEI check digit validation.

Term / Abbreviation	Full Form	Definition
JWT	JSON Web Token	A compact, self-contained token format (RFC 7519) for securely transmitting authentication claims. Used for AutoTal's mobile REST API.
Journal Voucher	—	An accounting document for adjustments, corrections, and opening balance entries not arising from purchase or sale transactions.
Ledger	—	**Business:** A book (or digital record) of all financial transactions organised by account. **Technical:** A row in AutoTal's LEDGER table representing one named account (e.g., Sales A/c, Axis Bank A/c).
LIFO	Last-In, First-Out	Inventory costing method where most recently received units are costed first on sale. **Not used in AutoTal.**
Luhn Algorithm	—	A checksum formula (ISO/IEC 7812) validating IMEI numbers. The 15th digit is the Luhn check digit computed from the preceding 14 digits.
MTD	Month-to-Date	Period from the first day of the current calendar month to today.
mylava	—	The retailer-facing portal operated by Lava International for sales order placement and stock checks.
NF	Normal Form	A stage in relational database normalisation (1NF, 2NF, 3NF, BCNF) reducing data redundancy and improving integrity.
1NF	First Normal Form	All column values are atomic; all rows are unique; no repeating groups exist.

Term / Abbreviation	Full Form	Definition
2NF	Second Normal Form	Meets 1NF; all non-key attributes fully depend on the entire primary key (no partial dependencies).
3NF	Third Normal Form	Meets 2NF; no non-key attribute is transitively dependent on the primary key.
OAuth 2.0	Open Authorization 2.0	Industry-standard protocol (RFC 6749) for delegated authorization. Used for Google Drive backup authentication.
ORM	Object-Relational Mapper	A software layer mapping database rows to programming language objects. AutoTal uses SQLAlchemy (Python).
P&L	Profit and Loss	Financial statement summarising revenues, costs, and expenses to show net profit or loss. Also called the Income Statement.
PAN	Permanent Account Number	10-character alphanumeric identifier issued by the Indian Income Tax Department to every taxpayer. Embedded in GSTIN.
PBKDF2	Password-Based Key Derivation Function 2	Cryptographic key stretching function (RFC 2898) deriving an encryption key from a passphrase using salt and many iterations. Used for AES-256 backup encryption key derivation in AutoTal.
PK	Primary Key	A column (or minimal set of columns) uniquely identifying each row in a table. Cannot be NULL.
Primary Stock	—	Mobile handset units received directly from Lava International via a Purchase Voucher. `stock_type = PRIMARY`.

Term / Abbreviation	Full Form	Definition
RBAC	Role-Based Access Control	A security model where system permissions are associated with roles (ADMIN, ACCOUNTANT) and users are assigned to roles.
Reconciliation	—	The process of comparing two independent sets of financial records (e.g., bank statement vs. accounting ledger) to verify consistency and identify discrepancies.
REST	Representational State Transfer	An architectural style for stateless, resource-oriented web APIs using standard HTTP methods (GET, POST, PUT, DELETE).
ROUND_HALF_UP	—	A rounding mode where a value exactly halfway between two options rounds toward the larger magnitude (e.g., 2.345 $\rightarrow$ 2.35). Used for all financial rounding in AutoTal.
Secondary Sales	—	Sales of mobile handsets from the distributor to retailers. Trigger Sale Voucher creation and reduce Primary Stock.
Secondary Stock	—	Units returned by retailers to the distributor via a Return/Credit Voucher. `stock_type = SECONDARY`.
SNR	Serial Number	Digits 9–14 of an IMEI (6 digits), assigned sequentially by the manufacturer to uniquely identify each device of a given TAC.
Specific Identification	—	Inventory valuation method where the exact purchase_cost of each individual IMEI is used as COGS when that unit is sold. No averaging or approximation.

Term / Abbreviation	Full Form	Definition
SQL	Structured Query Language	Standard language for managing and querying relational databases.
SQLAlchemy	—	Open-source Python ORM supporting PostgreSQL and SQLite. Used as AutoTal's Data Access Layer.
SRS	Software Requirements Specification	A formal document describing the purpose, environment, and requirements of a software system, per IEEE Std 830-1998.
TAC	Type Allocation Code	First 8 digits of an IMEI, allocated by GSMA, identifying the device manufacturer and model.
TLS	Transport Layer Security	Cryptographic protocol (RFC 8446) providing network communication security. AutoTal uses TLS 1.2+ for mobile API, TLS 1.3 for cloud backup.
Trial Balance	—	A list of all general ledger accounts with debit or credit balances at a given date. Verifies that total debits equal total credits.
UML	Unified Modelling Language	A standardised visual language for modelling software systems (class, sequence, and use case diagrams).
UTC	Coordinated Universal Time	Primary international time standard. IST = UTC+5:30. AutoTal stores timestamps in UTC and displays in IST.
UUID	Universally Unique Identifier	A 128-bit identifier (RFC 4122) generated to be globally unique without central coordination. Used as PK for Vouchers in AutoTal.

Term / Abbreviation	Full Form	Definition
Voucher	—	**Business:** A document recording a financial transaction (sale, purchase, payment, journal). **Technical:** A row in AutoTal's VOUCHER table plus one-or-more child TRANSACTION rows implementing the double-entry.
WAL	Write-Ahead Logging	A database durability technique where changes are written to a log file before being applied to main data files, enabling crash recovery without data loss.
XLS	Excel Spreadsheet (Legacy)	Microsoft Excel 97-2003 binary format (.xls). Supported for bank statement imports.
XLSX	Excel Open XML Spreadsheet	Microsoft Excel Open XML format (.xlsx), conforming to ISO/IEC 29500. Preferred import format.
YTD	Year-to-Date	Period from the beginning of the current financial year (April 1) to today's date.

10 Appendix B: Analysis Models

10.1 Data Flow Diagrams

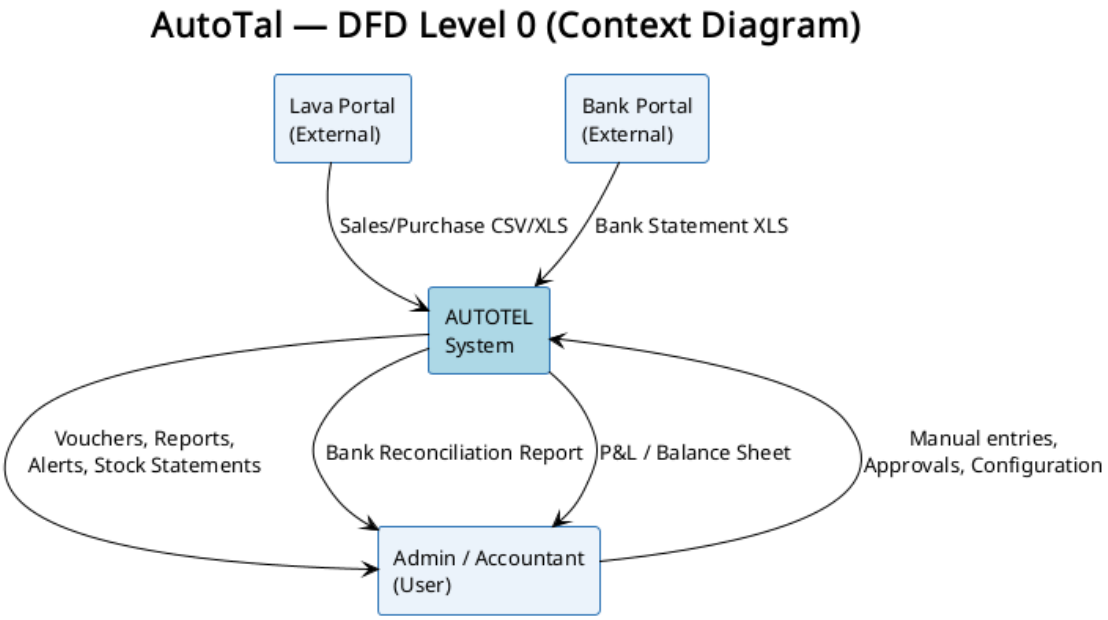


Figure 14: DFD Level 0 (Context Diagram)

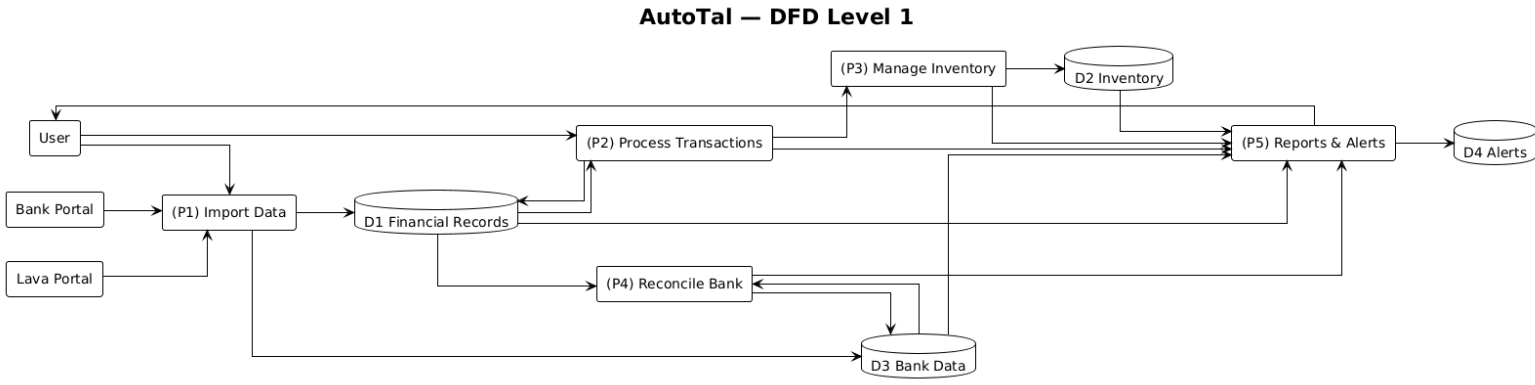


Figure 15: DFD Level 1

DFD, ER diagrams, UML diagrams.

10.2 Database Design

10.2.1 Entity Relationship Diagram

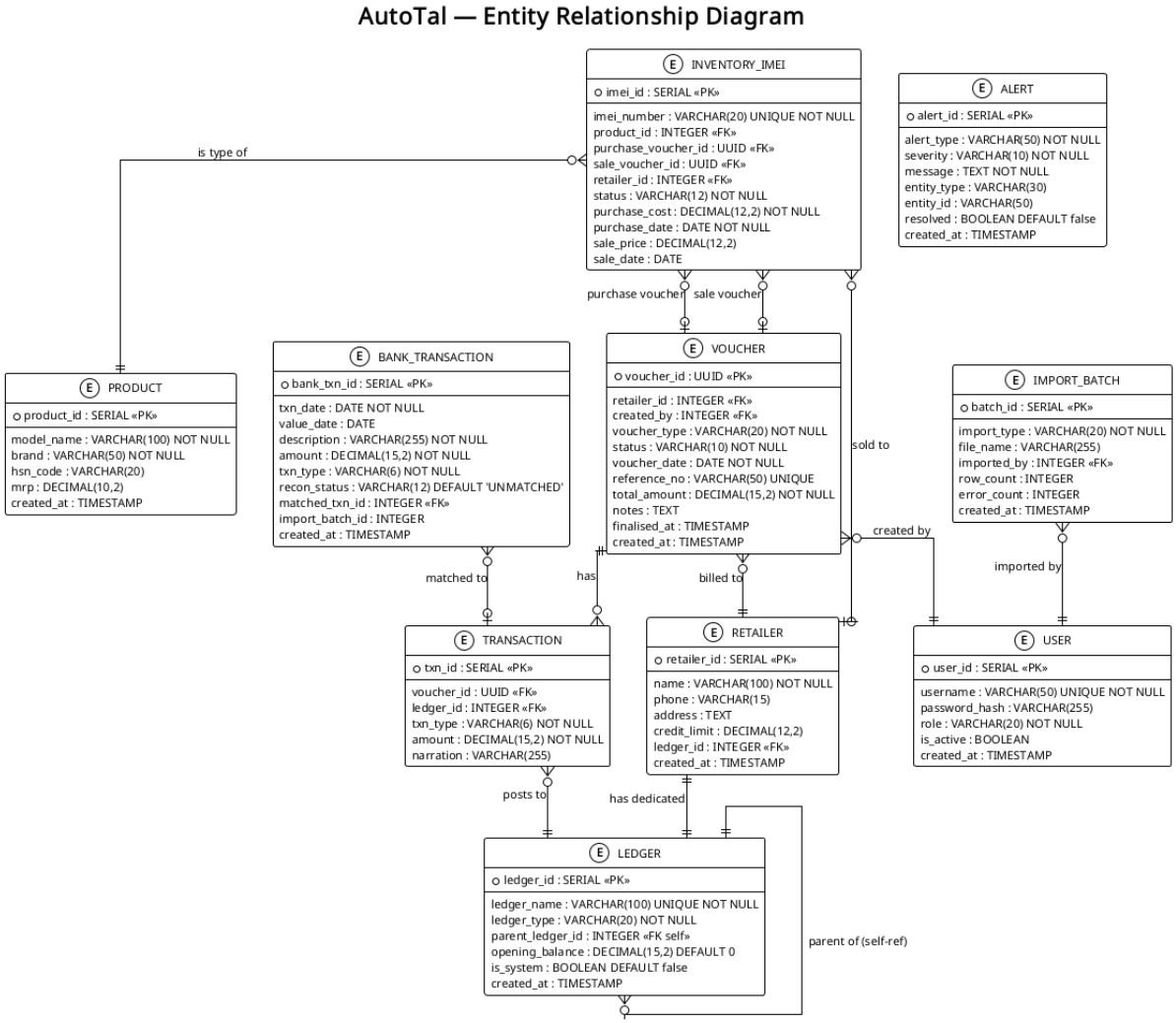


Figure 16: Entity-Relationship Diagram

10.2.2 Schema Tables

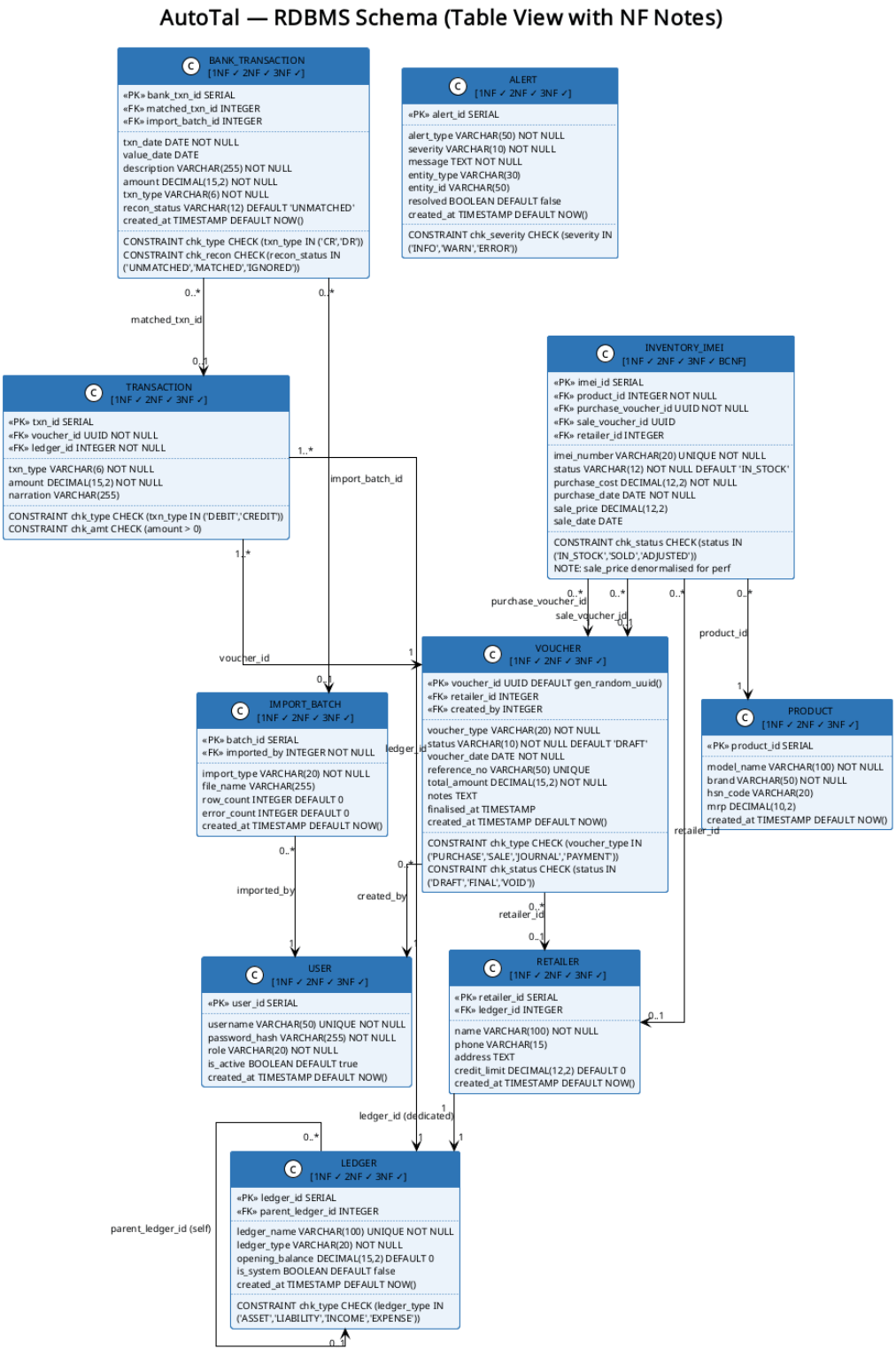


Figure 17: Database Schema Tables

10.3 Data Validation Specification

10.3.1 Lava Sales Import Schema

Field	Type	Format	Req'd	Validation Rule
date	DATE	YYYY-MM-DD	Yes	ISO 8601; not in future
imei_number	VARCHAR(20)	15 decimal digits	Yes	Luhn checksum; unique in system
model_name	VARCHAR(100)	Alphanumeric	Yes	Non-empty; must exist in PRODUCT table
retailer_code	VARCHAR(50)	Alphanumeric	Yes	Must match RETAILER.retailer_code
amount	DECIMAL(12,2)	Positive	Yes	$>$ 0.00; max 9,999,999.99
reference_no	VARCHAR(50)	Alphanumeric, hyphens	Yes	Non-empty; globally unique
hsn_code	VARCHAR(20)	Numeric	No	4–8 digit HSN code if provided
gst_rate	DECIMAL(5,2)	0.00–28.00	No	Valid GST rate: 0, 5, 12, 18, or 28
narration	VARCHAR(255)	Free text	No	Max 255 characters

10.3.2 Bank Statement Import Schema

Field	Type	Format	Req'd	Validation Rule
txn_date	DATE	YYYY-MM-DD	Yes	ISO 8601; not in future
value_date	DATE	YYYY-MM-DD	No	If present, $\geq$ txn_date
description	VARCHAR(255)	Free text	Yes	Non-empty
debit	DECIMAL(15,2)	Positive	No	$\geq$ 0; mutually exclusive with credit $>$ 0
credit	DECIMAL(15,2)	Positive	No	$\geq$ 0; mutually exclusive with debit $>$ 0
balance	DECIMAL(15,2)	Any	No	Running balance for cross-check

10.4 Error Code Registry

Error Code	Module	Description
ERR-IMP-001	Import	File size exceeds 10 MB limit
ERR-IMP-002	Import	Unsupported file format
ERR-IMP-003	Import	No valid rows found after validation
ERR-IMP-004	Import	Header row not detected in file

ERR-VCH-001	Voucher	Voucher not balanced (DR ≠ CR)
ERR-VCH-002	Voucher	Attempt to edit a finalised voucher
ERR-VCH-003	Voucher	Duplicate reference number
ERR-INV-001	Inventory	IMEI already SOLD
ERR-INV-002	Inventory	IMEI not found in inventory
ERR-INV-003	Inventory	Duplicate IMEI insertion
ERR-DB-001	Database	Database connection failure
ERR-REP-001	Reports	Stock formula mismatch detected
WARN-IMP-001	Import	Duplicate reference number (advisory)
WARN-RET-001	Retailer	Credit limit exceeded (advisory)
WARN-INV-001	Inventory	Inventory count mismatch

10.5 Requirement Traceability Matrix (Sample)

Req. ID	Feature / Module	Test Case ID	Priority
REQ-IMP-1	File format validation	TC-IMP-001	High
REQ-IMP-2	Field validation (IMEI, Date, Amt)	TC-IMP-003 to 010	High
REQ-IMP-5	Duplicate detection	TC-IMP-011	High
REQ-SYNC-3	Voucher lifecycle enforcement	TC-VCH-001 to 005	High
REQ-SYNC-4	Double-entry balance rule	TC-VCH-006	High
REQ-SYNC-6	Atomic finalisation	TC-VCH-007	High
REQ-INV-2	IMEI Luhn validation	TC-INV-001	High
REQ-INV-4	Specific Identification valuation	TC-INV-005	High
REQ-BANK-2	Auto-match algorithm	TC-BANK-003	High
REQ-ACC-2	Account lockout (5 attempts)	TC-ACC-004	High
REQ-BACKUP-2	Backup AES-256 encryption	TC-BCK-002	High
REQ-ERR-2	Transaction rollback on failure	TC-ERR-001	High
REQ-INT-5	DECIMAL precision in computation	TC-INT-003	High

11 Appendix C: Future Enhancements

- OCR integration – Optical Character Recognition (OCR) for scanning physical delivery challans and invoices as an alternative import method (planned Version 3.0).

- Cloud sync – Cloud-first deployment with PostgreSQL hosted on VPS/cloud (AWS RDS, Azure Database for PostgreSQL) enabling multi-user concurrent access (planned Version 4.0).
- Direct Tally ERP write-back via Tally XML import/export API (planned Version 3.0).
- Multi-company support – manage multiple distributor entities under a single AutoTal installation with full data isolation (planned Version 3.0).
- GST return filing integration – auto-populate GSTR-1 and GSTR-3B from Sales Voucher data (planned Version 4.0).
- WhatsApp/SMS alert integration for daily summary reports and critical alerts (planned Version 4.0).